Local and global geographies. Shell companies in Luxembourg (1929-2016)

Benoît Majerus

Centre for Contemporary and Digital History, University of Luxembourg

Lars Wieneke

Institution

Demival Vasques Filho

Centre for Contemporary and Digital History, University of Luxembourg

©<AUTHOR or ORGANIZATION / FUNDER>. Published by De Gruyter in cooperation with the University of Luxembourg Centre for Contemporary and Digital History. This is an Open Access article distributed under the terms of the Creative Commons Attribution License CC-BY

©<AUTHOR or ORGANIZATION / FUNDER>. Published by De Gruyter in cooperation with the University of Luxembourg Centre for Contemporary and Digital History. This is an Open Access article distributed under the terms of the Creative Commons Attribution License CC-BY-NC-ND

In [ ]:
```
from IPython.display import Image, display

display(Image(("./media/placeholder.png"))
```

(optional) This article was orginally published (...)

FirstKeyword, SecondKeyword, AlwaysSeparatedByAComma

Comment Area:
[Demi: Do we need maps to support the decentralization claim above?. 09/09/2025 - Benoît will fill the counts in the google spreadsheet]

Shell companies are a cornerstone of 20th-century global capitalism, yet their historical trajectories and infrastructural roles remain understudied. Luxembourg, a pivotal offshore financial center since the 1920s, exemplifies how these entities bridge local geographies and transnational tax networks, but systematic analyses of their evolution are lacking. Here, we

combine digital methods with archival research to reconstruct the development of Luxembourg's shell company sector from 1929 to 2016, leveraging public company registers to map their spatial and temporal dynamics.

We reveal two key patterns: locally, shell companies transitioned from hyper-concentration around Boulevard Royal in the 1930s—where banks and notaries dominated domiciliation—to a decentralized model by the 2010s, expanding to districts like Kirchberg as specialized fiduciaries displaced traditional actors. Globally, Luxembourg's networks shifted from early reliance on Switzerland to partnerships with emerging hubs like the British Virgin Islands (1980s) and Ireland (1990s), with case studies of Panama and Niue illustrating adaptive integration into evolving tax chains. Crucially, we identify "hidden helpers"—notaries, lawyers, and fiduciaries—whose intermediary roles sustained these networks despite regulatory changes.

Methodologically, we innovate by applying computational techniques (GIS mapping, network analysis, and NLP-driven entity extraction) to fragmented, multilingual registers (50,000+ documents), enabling large-scale analysis of ~2.9 million business entries while addressing biases in digitized historical data. Our findings challenge narratives centered solely on banks, demonstrating how shell companies, as infrastructural tools, reflect Luxembourg's interplay of sovereignty, European integration, and global finance.

This study advances debates on offshore finance by unifying local and global scales of analysis, revealing the resilience of shell company infrastructures amid regulatory pressures like the EU's "Unshell" directive. It also underscores digital history's potential—and limitations—for critiquing financial opacity, offering a model for interdisciplinary research at the nexus of economic history, geography, and data science.

Introduction

Comment Area:
[Demi: do we need a citation for "Unshell"?
09/09/2025 - Benoît will send it to Demi]

[Narrative layer]
In December 2021, the European Commission presented the "Unshell" Directive, designed to combat the misuse of shell entities for tax avoidance within the European Union ("Unshell") (Sinnig and Zetzsche 2023). This measure was introduced in the context of significant strain on public finances during the COVID-19 pandemic, which saw a substantial increase in public deficits across most European states. However, this directive is part of a broader, long-term effort in global tax governance to curb various forms of tax evasion. A transnational governance of taxation has been in the making since the interwar period, at the latest (Farquet 2010), but moments of intense development were followed by long periods of inactivity. The Unshell directive is part of a longer sequence that started after the financial

crisis that hit the world starting in 2007, on the one hand, and by the scandalization of certain practices by journalist consortia, such as the Panama Papers, LuxLeaks, or Cyprus Confidential, on the other hand. The OECD and G20 committed to create an international framework to combat tax avoidance by multinational enterprises, starting from 2013 through the Base Erosion and Profit Shifting (BEPS) Action Plan. Since 2016, the European Union has launched several Anti-Tax Avoidance Packages (ATAD). The Unshell directive is part of a third component of ATAD (Sinnig and Zetzsche 2023). At the time of publishing this article, the Unshell directive is still navigating through the labyrinths of the infamous EU trilogue. Subsequently, the transposition of the directive into each member country will reveal its true implications for the tax chains, with Luxembourg sometimes taking a long time to transpose directives that seem unfavorable to its financial center (Bourbaki 2016). Regardless of the fate of the Unshell directive, it has brought to the fore a tool for tax planning and beneficial ownership avoidance: shell companies.

[Narrative layer]
Shell companies are difficult to define: most authors agree that a common trait is the absence of any economic substance and that they are an essential part of several tax havens (Beckett 2023). Be it Switzerland, Luxembourg or the British Virgin Islan, all of them introduced legal structuring of companies that permitted tax reduction and anonymisation, the first two through the holding regime that became a “refuge for capital flight” (Paquier 2001, 179) in the interwar period or through the International Business Company Act in the 1990s for the BVI (Maurer 2000). But till now, they have not attracted much attention till now, one exception being the recent (Weitzman 2022). Shell companies are nonetheless an essential infrastructure in global tax chains: most of the offshore centers offer this legal construct and so does Luxembourg. Although the Grand Duchy of Luxembourg was not among the first generation of tax havens (Guex 2022; Watteyne 2023), by the late 1920s it had adopted a legal structure—the holding company—that positioned it in the market for tax engineering and the anonymization of actual beneficiaries (Calabrese and Majerus 2023).

[Narrative layer]
Based on the publicly available company registers, this article maps the shell companies as an essential infrastructure of the Luxembourg financial offshore center by analyzing how they were inscribed in the local geography of the capital of the Grand-Duchy and how they linked to other offshore center in larger global tax chains. Geographers have been studying the geography of the Luxembourg financial center for some time now, focusing on its local dynamics as well as its role in global financial geography (Walther, Schulz, and Dörry 2011; Dörry 2016; Hesse and Wong 2019), although they have not greatly emphasized the historical dimension of this process. Historians have so far scarcely explored this topic, focusing mainly on banks (Pauly 2018; Duval, Gabellini, and Mouton 2023). While banks are certainly significant actors, they are not the only financial service providers. The establishment of shell companies, whose headquarters are often located in the offices of service providers, allows for a parallel or complementary geography of the financial sector.

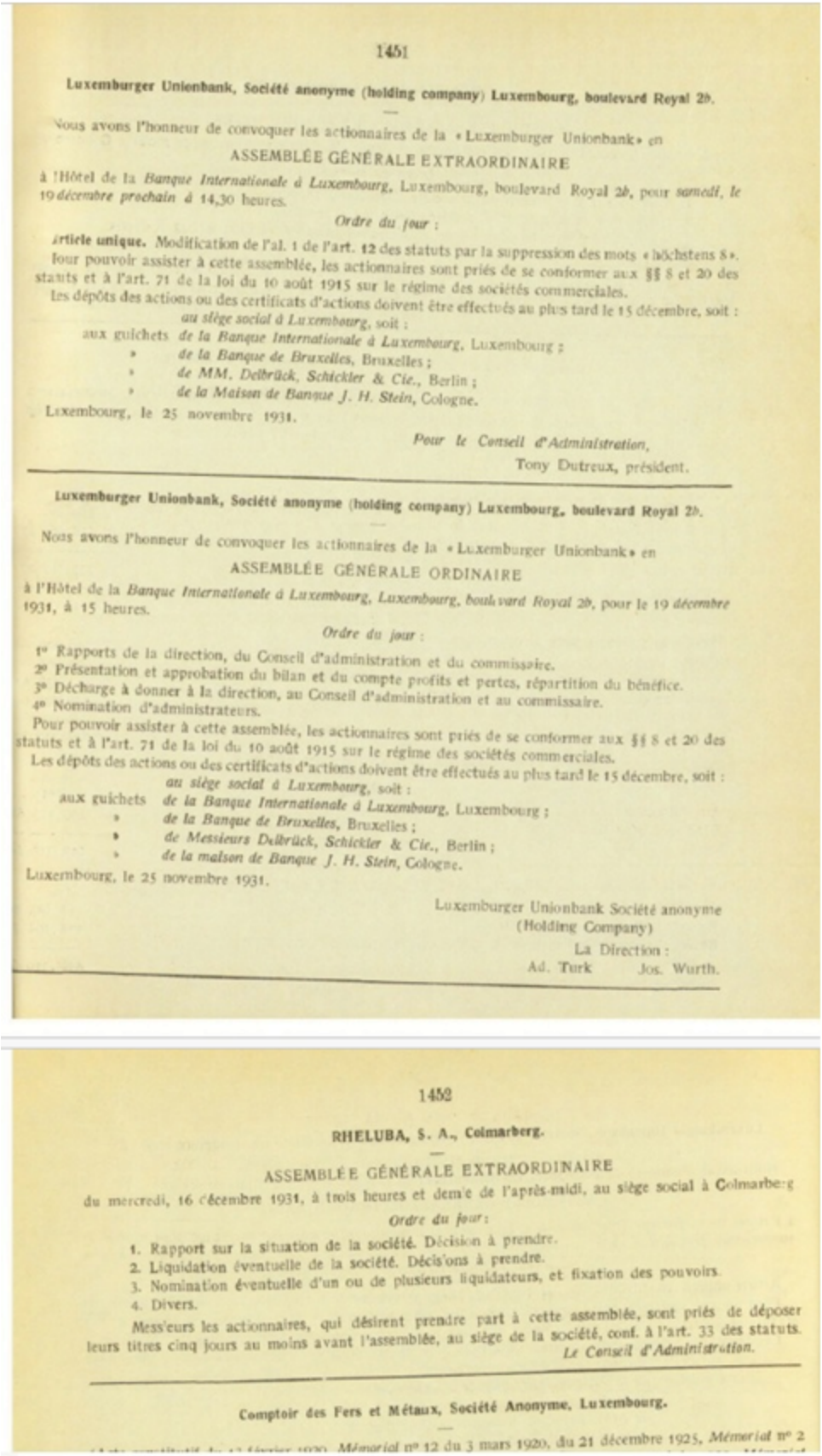
Sources

[Narrative layer]
In the midst of World War I, Luxembourg reformed its corporate regulation with the law of August 10, 1915. ('Loi Du 10 Août 1915 Concernant Les Sociétés Commerciales' 1915). This law required every company to make public certain information for each company established in Luxembourg. Published as an appendix to the Memorial, the official journal of the Grand Duchy of Luxembourg, these texts were entitled, from 1960 onwards, to a separate publication, Mémorial C ('Règlement Grand-Ducal Du 9 Janvier 1961 Relatif Aux Trois Recueils Du Mémorial', n.d.).

[Hermeneutics layer]
Unfortunately the corpus does not exist under a digitised, coherent and semi-structured format. The years 1929-1960 were scanned at the Luxembourg Centre for Contemporary and Digital History (C2DH) on a Treventus ScanRobot 2.0 MDS book scanner (400 dpi in tif format). Thus, Optical Character Recognition (OCR) plays a critical role in extracting text from scanned documents. However, during our work, we encountered several challenges that significantly impacted the accuracy and reliability of the extracted text.

[Hermeneutics layer]
One of the primary issues was related to the layout of the scanned documents. The pages contained white margins that acted as long 2D separators, inadvertently segmenting business descriptions. This disrupted the logical flow of the text and led to incorrect parsing. Additionally, some pages were distorted and skewed, tilting either left or right, which further complicated text extraction and alignment (Fig. 1). These challenges underscore the complexities associated with OCR processing when dealing with inconsistent document layouts and distortions. To deal with this problem programmatically, it was necessary to implement advanced preprocessing techniques such as margin removal, deskewing algorithms, and language-specific OCR models. These enhancements help mitigate errors and ensure more reliable text extraction in the OCR application. However, since the number of mentioned businesses in the documents for this period was not considerable, and the above-mentioned methods require significant time and effort to implement, we decided to process this time period manually.

In [1]:
```
from IPython.display import Image, display

display(Image("./media/source-challenge.png"))
```



1451

Luxemburger Unionbank, Société anonyme (holding company) Luxembourg, boulevard Royal 2b.

Nous avons l'honneur de convoquer les actionnaires de la « Luxemburger Unionbank » en

ASSEMBLÉE GÉNÉRALE EXTRAORDINAIRE

à l'Hôtel de la *Banque Internationale à Luxembourg*, Luxembourg, boulevard Royal 2b, pour *samedi, le 19 décembre prochain à* 14,30 heures.

Ordre du jour :

Article unique. Modification de l'al. 1 de l'art. 12 des statuts par la suppression des mots « höchstens 8 ».

Pour pouvoir assister à cette assemblée, les actionnaires sont priés de se conformer aux §§ 8 et 20 des statuts et à l'art. 71 de la loi du 10 août 1915 sur le régime des sociétés commerciales.

Les dépôts des actions ou des certificats d'actions doivent être effectués au plus tard le 15 décembre, soit :

au siège social à Luxembourg, soit :
aux guichets *de la Banque Internationale à Luxembourg*, Luxembourg ;
» *de la Banque de Bruxelles*, Bruxelles ;
» *de MM. Delbrück, Schickler & Cie.*, Berlin ;
» *de la Maison de Banque J. H. Stein*, Cologne.

Luxembourg, le 25 novembre 1931.

Pour le Conseil d'Administration,
Tony Dutreux, président.

Luxemburger Unionbank, Société anonyme (holding company) Luxembourg, boulevard Royal 2b.

Nous avons l'honneur de convoquer les actionnaires de la « Luxemburger Unionbank » en

ASSEMBLÉE GÉNÉRALE ORDINAIRE

à l'Hôtel de la *Banque Internationale à Luxembourg, Luxembourg, boulevard Royal 2b*, pour le 19 *décembre* 1931, à 15 heures.

Ordre du jour :

1° Rapports de la direction, du Conseil d'administration et du commissaire.
2° Présentation et approbation du bilan et du compte profits et pertes, répartition du bénéfice.
3° Décharge à donner à la direction, au Conseil d'administration et au commissaire.
4° Nomination d'administrateurs.

Pour pouvoir assister à cette assemblée, les actionnaires sont priés de se conformer aux §§ 8 et 20 des statuts et à l'art. 71 de la loi du 10 août 1915 sur le régime des sociétés commerciales.

Les dépôts des actions ou des certificats d'actions doivent être effectués au plus tard le 15 décembre, soit :

au siège social à Luxembourg, soit :
aux guichets *de la Banque Internationale à Luxembourg*, Luxembourg ;
» *de la Banque de Bruxelles*, Bruxelles ;
» *de Messieurs Delbrück, Schickler & Cie.*, Berlin ;
» *de la maison de Banque J. H. Stein*, Cologne.

Luxembourg, le 25 novembre 1931.

Luxemburger Unionbank Société anonyme
(Holding Company)
La Direction :
Ad. Turk Jos. Wurth.

1452

RHELUBA, S. A., Colmarberg.

ASSEMBLÉE GÉNÉRALE EXTRAORDINAIRE

du mercredi, 16 décembre 1931, à trois heures et demie de l'après-midi, au siège social à Colmarberg

Ordre du jour :

1. Rapport sur la situation de la société. Décision à prendre.
2. Liquidation éventuelle de la société. Décisions à prendre.
3. Nomination éventuelle d'un ou de plusieurs liquidateurs, et fixation des pouvoirs.
4. Divers.

Messieurs les actionnaires, qui désirent prendre part à cette assemblée, sont priés de déposer leurs titres cinq jours au moins avant l'assemblée, au siège de la société, conf. à l'art. 33 des statuts.

Le Conseil d'Administration.

Comptoir des Fers et Métaux, Société Anonyme, Luxembourg.

Mémorial n° 12 du 3 mars 1920, du 21 décembre 1925, *Mémorial* n° 2

[Figure] **Fig. 1.** Distorted and skewed, tilting either left or right, scanned pages

[Hermeneutics layer]

Moving forward, the years 1961-1995 were digitised by the National Library of Luxembourg

(BNL). Thanks to a convention with the BNL, our research project got access to these files that are publically available on e-luxemburgensia[^1] but with major restrictions. For the last remaining decades (1996-2016), we downloaded the pdf files from legilux.public.lu[^2]. Although these scanned documents were of relatively high quality, challenges remained in extracting text using OCR. Notably, the multilingual nature of the documents—written in French, German, and Luxembourgish—introduced additional complications. Accented characters such as é, ä, ü, and ö were frequently misrecognized, leading to character-level errors. Punctuation marks also caused ambiguities: for example, a period following a capital letter (e.g., "I.") was sometimes misread as "L", while "O." was incorrectly identified as "Q". These challenges affected the quality of information extracted from the scanned documents. To improve accuracy, we employed generative AI (Large Language Models—LLMs), combined with human review, as part of a post-cleaning and information enhancement process to mitigate the impact of these issues on the extracted business descriptions. It's worth noting that, despite these refinements, the dataset contains approximately 3 million business descriptions from 1961 to 2016. Assuming an average of 30 seconds per post-cleaning task, it would take roughly three years to process the entire dataset. Therefore, some degree of inaccuracy remains and is expected to be gradually corrected in the coming years.

[footnotes]
[^1]: "The digitized version of the Memorial C (1915-1940, 1960-1995), which also contains personal data, is not subject to indexing and search, except for the titles of the articles, in compliance with the GDPR. The PDF files of the various issues, articles, and pages cannot be downloaded by users." translated by the authors https://eluxemburgensia.lu/periodicals/memorialc (last accessed: 16 February 2024)
[^2]: Unfortunately the Luxembourgish administration refused to give us access to the structured data they possess for the years 1996-2016: we had only access to the 'unstructured' pdfs.

[Hermeneutics layer]

Comments Area:
[Demi: What about 1960?
09/09/2025 - Benoît will check where the data from 1960 comes from]

In summary, the register of companies of Luxembourg is publicly available: for the years 1940 to 1959 on paper in some Luxembourgish libraries, for the years 1929 to 1939 and 1961 to 2016 on the web. The policies of publicity are however different: paradoxically the BNL does protect the older data (1929-1939, 1961-1996) more strictly than the Prime Minister administration, responsible for the website legilux.public.lu that gives you access to the years 1996-2016.

[Narrative layer]
The register of companies does not fall under the restriction of the Luxembourgish Archival

Law as it is publicly available from the day it is published. But as data is collected, analyzed, stored and processed, the General Data Protection Regulation's (GDPR) that has been implemented in the European Union since 2018, applies. If the GDPR explicitly provides for exceptions for research, the wording remains rather vague. While there is today a growing body of case law concerning certain data collections and publications - notably for services such as online tracking (offered by, for example, Google) or regarding the work of journalists (Bitiukova 2023), historians have relatively little formalized their practices, apart from the obligations they may face from those who hold the data.

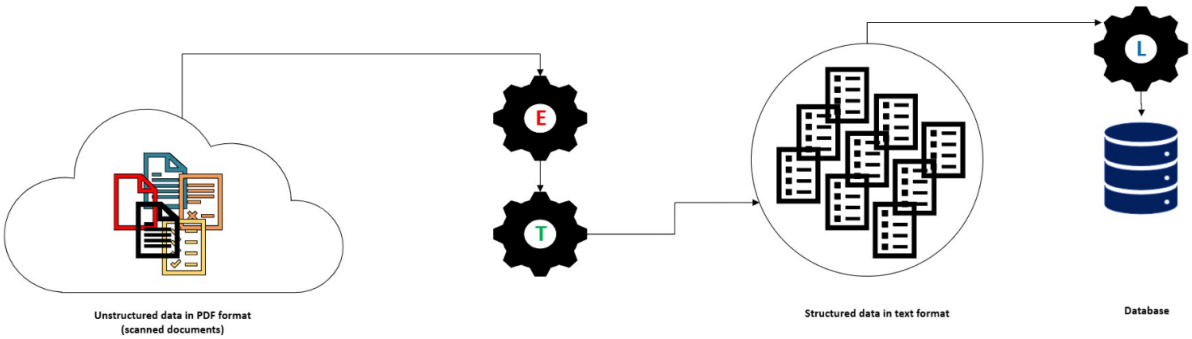
[Narrative layer]
Some of the solutions proposed in the literature are inapplicable in this project. Obtaining authorization of the mentioned names (around 350,000 persons in our database) would require a disproportionate effort. Pseudonymization or anonymization during data collection but also during publication is contrary to the goal of the research, which specifically aims to identify the manufacturing chains of shell companies and to historicize the actors and uncover the societal networks within which they operated (Luyten 2022; Friedewald forthcoming). We will therefore, if necessary for the historical argument, publish the name of financial service providers such as notaries, lawyers or bankers. If the "Indeterminate legal concepts" and the "large scopes of interpretation" (Jahnel 2023, 561–62) of the GDPR allows such a practice we inserted nevertheless on the webpage of the project a "your section" allowing people to access and correct information we have collected.

Data forging: the ETL Workflow for the historical records

[Hermeneutics layer]
In this study, we process unstructured data in PDF format (scanned documents) through an ETL (Extract, Transform, Load) workflow (Fig. 2). The process involves extracting data using 2D line object detection, transforming it into a structured text format with the help of LLMs, and loading it into a relational database specifically design for these data. In the following subsections, we detail each step of this workflow.

In [1]:
```
#from IPython.display import Image, display

#display(Image("./media/Fig T2.png"))
```

In [2]:
```
from IPython.display import Image, display

display(Image("./media/etl-schematic.png"))
```



[Figure] **Fig. 2.** Schematic drawing of a standard ETL (Extract, Transform, Load) workflow used to convert unstructured data into structured data and make it available in a dedicated database. In this research work, Extract involved reading scanned documents and capturing business descriptions for each business; Transform involved applying Named Entity Recognition (NER); and Load involved saving the structured NER output and business descriptions into a database.

Extract

In [ ]:
```
!python.exe -m pip install --upgrade pip
!pip install pytesseract
!pip install opencv-python
!pip install pdfplumber
!pip install peft
!pip install trl
!pip install tensorboard
!pip install tensorboardX
```

[Hermeneutics layer]
In the Extract step of the ETL workflow, unstructured PDF documents are processed to extract text data using a combination of Optical Character Recognition (OCR) and advanced image preprocessing techniques. Each page of the PDF is first converted into an image using the pdfplumber library (Singer-Vine and Jain 2025). These images are then preprocessed by converting them to grayscale and applying binary thresholding to enhance text visibility. Line detection uses morphological operations to identify horizontal structures, which segment the image into manageable regions. The segmented areas are processed using Tesseract OCR (Hoffstaetter 2025), configured with multilingual support for languages such as French, German, Luxembourgish, and English, to obtain high-quality textual data. The extracted content is subsequently refined and organized into coherent blocks based on line and paragraph alignment, ensuring improved readability and accuracy. Finally, the structured text is aggregated and saved to a text file, preparing it for the subsequent transformation and loading phases. An explanation of the parameters and components used in the extraction process is outlined below. The script begins by importing essential libraries for file handling (os), image processing (cv2, PIL), Optical Character Recognition (pytesseract), PDF parsing (pdfplumber), and data manipulation (pandas, numpy). The Tesseract OCR engine is

configured by specifying its executable path, and multilingual support is verified via subprocess to list installed languages.

Input and output paths are defined to manage the location of PDF files and the resulting extracted text. The core extract_text function performs preprocessing steps such as grayscale conversion and binary thresholding to improve OCR accuracy. Tesseract is then used to extract text in multiple languages (French, German, Luxembourgish, and English), organizing content into structured text blocks with attention to alignment and spacing.

The main processing loop iterates through each PDF in the input directory, converting pages into images and applying morphological operations to detect horizontal lines. These lines are used to segment the page into distinct regions, from which text is extracted and formatted. The refined text from all pages is aggregated and saved as a single structured text file. For validation purposes, matplotlib is used to visualize both the original pages and cropped sections during processing. The final output includes the structured text and a log of the processing status for each file.

In [ ]:
```
import os
import cv2
import pytesseract
import pdfplumber
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt  # For displaying images
from pytesseract import Output
import pandas as pd
import subprocess

# Configure Tesseract executable path for Windows
pytesseract.pytesseract.tesseract_cmd = r'C:\Program Files\Tesseract-OCR\tesseract.

# List installed languages to verify Tesseract setup
result = subprocess.run([pytesseract.pytesseract.tesseract_cmd, '--list-langs'], st
print("Installed Tesseract Languages:\n", result.stdout)

# Path where processed files will be saved
pathW = "\\JDH_PAPER\\"  # Adjust path to your environment

def extract_text(image):
    """
    Extract text from an image using Tesseract OCR.

    Args:
        image (numpy.ndarray): Input image from which text will be extracted.

    Returns:
        str: Extracted text.
    """
    try:
        # Convert the image to grayscale
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    except Exception as e:
```

```
        gray = image
        print(f"Error during grayscale conversion: {e}")

    try:
        # Apply binary thresholding to enhance text contrast
        thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTS
    except Exception as e:
        print(f"Error during thresholding: {e}")
        thresh = image

    # Configure Tesseract for multilingual OCR
    custom_config = r'-c preserve_interword_spaces=1 --oem 1 --psm 6 -l fra+deu+ltz

    # Extract OCR data from image
    d = pytesseract.image_to_data(thresh, config=custom_config, output_type=Output.
    df = pd.DataFrame(d)
    df1 = df[(df.conf != '-1') & (df.text != ' ') & (df.text != '')]

    # Sort text blocks vertically
    sorted_blocks = df1.groupby('block_num').first().sort_values('top').index.tolis
    text = ''
    for block in sorted_blocks:
        curr = df1[df1['block_num'] == block]
        sel = curr[curr.text.str.len() > 5]
        char_w = (sel.width / sel.text.str.len()).mean()
        prev_par, prev_line, prev_left = 0, 0, 0
        for ix, ln in curr.iterrows():
            # Add new line when switching paragraphs or lines
            if prev_par != ln['par_num']:
                text += '\n'
                prev_par = ln['par_num']
                prev_line = ln['line_num']
                prev_left = 0
            elif prev_line != ln['line_num']:
                text += '\n'
                prev_line = ln['line_num']
                prev_left = 0

            # Calculate space adjustments for alignment
            added = 0
            if ln['left'] / char_w > prev_left + 1:
                added = int((ln['left']) / char_w) - prev_left
                text += ' ' * added
            text += ln['text'] + ' '
            prev_left += len(ln['text']) + added + 1
        text += " \n"
    return text

# Process each year within a specific range
for year in range(1961, 1962):
    path = "\\JDH_PAPER\\"  # Adjust this path accordingly
    List = os.listdir(path)
    previous_end_crop = ""  # Stores the last crop of a page to combine with the ne

    for each in List:
        if each.endswith(".pdf"):
```

```
        print(f"Processing file: {each}")
        pdf_file = os.path.join(path, each)
        outputFilesPath = os.path.join(pathW, each.replace(".pdf", ".txt"))
        my_pdf = pdfplumber.open(pdf_file)

        y_all = {i: [] for i in range(len(my_pdf.pages))}
        All_Text = ""
        All_Text += "==========================================================
        All_Text += f"\n +++++++++++++++++ \n File name: {each} Page Number: {1

        for i in range(len(my_pdf.pages)):
            # Convert each page to an image
            im = my_pdf.pages[i].to_image(resolution=420)
            print(f"Processing page {i + 1}")
            im.save("\\JDH_PAPER\\temporary.png", "PNG")
            image = cv2.imread("\\JDH_PAPER\\temporary.png")

            # Display the original page
            plt.figure(figsize=(12, 10))
            plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
            plt.title(f"Original Page {i}")
            plt.axis('off')
            plt.show()

            # Preprocess for line detection
            gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
            thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.TH
            kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 1))
            dilated = cv2.dilate(thresh, kernel, iterations=2)
            horizontal_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (320,
            detected_lines = cv2.morphologyEx(dilated, cv2.MORPH_OPEN, horizont

            # Detect lines and extract their coordinates
            cnts = cv2.findContours(detected_lines, cv2.RETR_EXTERNAL, cv2.CHAI
            cnts = cnts[0] if len(cnts) == 2 else cnts[1]

            for c in cnts:
                y = c[0][0][1]
                if y > 0:
                    y_all[i].append(y)

            y_all[i] = sorted(list(set(y_all[i])))
            print("Detected line y-coordinates:", y_all[i])

            start = [0] + y_all[i] + [image.shape[0]]

            # Crop and process each region between detected lines
            for k in range(len(start) - 1):
                cropImage = image[start[k]:start[k + 1], :]
                plt.figure(figsize=(12, 10))
                plt.imshow(cv2.cvtColor(cropImage, cv2.COLOR_BGR2RGB))
                plt.title(f"Page {i} - Crop {k + 1}")
                plt.axis('off')
                plt.show()

                text = extract_text(cropImage)
```

```
                print("Extracted text:", text)

                if k == len(start) - 2 and i != len(my_pdf.pages) - 1:
                    previous_end_crop = text
                else:
                    All_Text += text
                    All_Text += "\n ==========================================
                    All_Text += f"\n +++++++++++++++++ \n File name: {each} Pag

            if previous_end_crop:
                All_Text += previous_end_crop
                previous_end_crop = ""

        # Save extracted text to file
        with open(outputFilesPath, "w") as outputFiles:
            outputFiles.write(All_Text)
        print("Processing completed for:", each)
```

Transform

[Hermeneutics layer]
In the transform phase of the ETL workflow, raw extracted data is cleaned, structured, and enriched to ensure it is in a usable format for analysis. For this paper, we utilized generative AI models, specifically Large Language Models (LLMs), to transform unstructured text into well-structured and semantically enriched formats, enabling more accurate and meaningful insights.

[Hermeneutics layer]
Now we explain a code that outlines a workflow to process a text file, extract valid company names using regex cleaning and a generative AI model, and save the results to an output file. It begins by importing essential libraries: transformers for using pre-trained models like Mistral, torch for GPU and model operations, re for pattern matching, and gc for memory cleanup. Two text-cleaning functions handle noise removal: clean_start_of_text_number1 removes leading characters and numbers, while clean_start_of_text_number2 removes text before two-digit numbers, accounting for short and uppercase formats. The Mistral model (microsoft/Orca-2-7b) and tokenizer are initialized with support for GPU and Hugging Face tokens. The validate_company_name function extracts and validates concise company names from descriptions using a structured prompt. extract_company_names generates names based on this prompt, cleans the output, and filters irrelevant cases, such as lines with "Siège social" or "Sitz." The main workflow reads the input file line by line, cleans the text, prepares prompts, extracts and validates names, and filters invalid results (e.g., returning "Wrong" for non-company content).Memory is optimized during GPU-intensive processing using torch.cuda.empty_cache and gc.collect. Finally, results are saved to extracted_company_names.txt, pairing extracted company names with their original input lines using a delimiter (&&&****&&&) for easy parsing and analysis.

In [ ]:
```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch,gc
import re

import re

def clean_start_of_text_number1(input_text):
    # Remove any punctuation from the first characters
    cleaned_text = re.sub(r'^[^a-zA-ZÀ-ÖØ-öø-ÿÄäÖöÜüßÇçÉéÈèÊêËëÀàÂâÎîÏïÔôÛûÙùŸÿÆæŒœ

    # Check if there's a number at the start of the text and remove everything befo
    match = re.search(r'^\s*\d+', cleaned_text)
    if match:
        # Remove the leading number and any preceding characters
        cleaned_text = cleaned_text[match.end():].strip()
    else:
        # If no number at the start, re-clean remaining leading junk characters
        cleaned_text = re.sub(r'^[^\wÀ-ÖØ-öø-ÿÄäÖöÜüßÇçÉéÈèÊêËëÀàÂâÎîÏïÔôÛûÙùŸÿÆæŒœ

    return cleaned_text

def clean_start_of_text_number2(input_text):
    # Find the first occurrence of a number with 2 or more digits and remove all te
    if len(input_text)<50:
        end1=len(input_text)
    else:
        end1 =35
        if input_text[:end1].isupper()==True:
            end1=0
    match = re.search(r'\b\d{2,}', input_text[:end1])
    if match:
        # Remove everything before and including the first occurrence of the number
        cleaned_text = input_text[match.end():].strip()
    else:
        # If no number with 2 or more digits is found, return the original input
        cleaned_text = input_text.strip()

    return cleaned_text

# Initialize the Mistral model and tokenizer
model_name = "microsoft/Orca-2-7b"#"Open-Orca/Mistral-7B-OpenOrca"
# model_name = "mistralai/Mistral-7B-Instruct-v0.2"

device = "cuda" if torch.cuda.is_available() else "cpu"

print("Device:", device)

tokenizer = AutoTokenizer.from_pretrained(model_name, use_auth_token='hf_HDQStUzyDx
model = AutoModelForCausalLM.from_pretrained(model_name, use_auth_token='hf_HDQStUz

def validate_company_name(company_description, suggested_company_name, model, token
    """
    Validates and extracts the correct company name based on the provided descripti

    Args:
        company_description (str): The company description containing possible comp
```

```
        suggested_company_name (str): The suggested full company name.
        model (AutoModelForCausalLM): Pretrained language model for causal generati
        tokenizer (AutoTokenizer): Tokenizer corresponding to the pretrained model.

    Returns:
        str: Extracted concise company name.
    """
    prompt = (
        "You are tasked with validating and extracting the company name from a desc
        "The company name may be written in full or as an abbreviation.\n\n"
        "Instructions:\n"
        "1. Identify the most concise version of the company name within the provid
        "2. If an abbreviation or alternate form of the name is explicitly stated,
        "3. Return only the exact company name as it appears in the text, without a
        "Example 1:\n"
        "Company Description: \"e OMNIUM INTERNATIONAL S.A.» en abréviation: « OMIN
        "Suggested Company Name: \"OMNIUM INTERNATIONAL S.A.\"\n"
        "Output: \"OMINTER\"\n\n"
        "Example 2:\n"
        "Company Description: \"GLOBALTECH INNOVATIONS GmbH, also referred to as 'G
        "Suggested Company Name: \"GLOBALTECH INNOVATIONS GmbH\"\n"
        "Output: \"GLOBTECH\"\n\n"
        "Example 3:\n"
        "Company Description: \"Alpha Corp. Official Name: 'Alpha Corporation'.\"\n
        "Suggested Company Name: \"Alpha Corp.\"\n"
        "Output: \"Alpha Corporation\"\n\n"
        "Company Description: ABC Société Anonyme. fabrique de cuivre, "
        "Suggested Company Name: \"ABC, Société Anonyme.  fabrique de cuivre,\"\n"
        "Output: \"ABC\"\n\n"
        "Now process the following:\n\n"
        f"Company Description: {company_description}\n"
        f"Suggested Company Name: {suggested_company_name}\n"
        f"Output:"
    )

    # Tokenize and generate response
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        output = model.generate(**inputs, max_new_tokens=50, temperature=0.2, do_sa

    # Decode the response and extract the result
    generated_text = tokenizer.decode(output[0], skip_special_tokens=True)
    extracted_name = generated_text.split("Output:")[-1].strip()
    return extracted_name

# Function to generate company name predictions
def extract_company_names(prompt, model, tokenizer):
    # Ensure inputs are on the same device as the model
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=200
    outputs = model.generate(**inputs, max_new_tokens=30, eos_token_id=tokenizer.eo
    output_text = tokenizer.decode(outputs[0], skip_special_tokens=True).replace("\
    if "company name:" in output_text:
        output_text=output_text.split("company name:")[1]
    if "Company name:" in output_text:
        output_text=output_text.split("Company name:")[1]
    if "company names:" in output_text:
```

```
        output_text=output_text.split("company names:")[1]
    if "Company names:" in output_text:
        output_text=output_text.split("Company names:")[1]
    if "Siège social" in output_text:
        output_text=output_text.split("Siège social")[0]
    if "Siege social" in output_text:
        output_text=output_text.split("Siege social")[0]
    if "Siége social" in output_text:
        output_text=output_text.split("Siége social")[0]
    if "Hauptsitz" in output_text:
        output_text=output_text.split("Hauptsitz")[0]
    if "Sitz" in output_text:
        output_text=output_text.split("Sitz")[0]
    if "sitz" in output_text:
        output_text=output_text.split("sitz")[0]
    if "S. à r." in output_text:
        output_text=output_text.split("S. à r.")[0]
    output_text = clean_start_of_text_number2(output_text)
    print("Output Text: ", output_text)
    return output_text


# Read the input file line by line
input_file = "/JDH_PAPER/1970-01-23_01_Single_Line.txt"  # Replace with your input
output_file = "/JDH_PAPER/extracted_company_names.txt"

with open(input_file, "r") as infile, open(output_file, "w") as outfile:
    for line in infile:
        line=line.strip().lstrip().rstrip()
        if len(line)>2:
            input_text=line.split('+++++++++++++++++')[-1].strip()
            if len(input_text)<130:
                end=len(input_text)
            else:
                end=130
            if "SOMMAIRE" in input_text[:20] or 'MEMORIAL Journal Officiel me Amtsb
                company_names="Wrong"
            else:
                input_text=input_text[:end]
                input_text=clean_start_of_text_number1(input_text)
                # input_text=clean_start_of_text(input_text)

                # Prepare the prompt for Mistral with clear instruction
                prompt = (
                    f"In the following text, company names are short and exact phra
                    f"If the text is too short or it is not part of a company descr
                    f"It is possible for the given input text to begin with irrelev
                    f"Extract the exact company names from the provided text withou
                    f"{input_text.strip()}\n\n"
                    f"Company Names:"
                )


                print("prompt:",prompt)
                print("==========")
                generated_text = extract_company_names(prompt, model, tokenizer)
```

```
                generated_text= generated_text.rstrip().lstrip().strip()
                company_names = generated_text
                del generated_text
                torch.cuda.empty_cache()
                gc.collect()
                result = validate_company_name(input_text.strip(), company_names, m
                company_names = result
                print("Final company_names:",company_names)
                print("**********************************")

                del result
                torch.cuda.empty_cache()
                gc.collect()
                # Filter valid company names

            # Write each company name to the output file
            outfile.write(company_names +" &&&****&&& "+line.replace("\n","")+ "\n"

print(f"Extraction completed. Company names are saved in extracted_company_names.tx
```

[Hermeneutics layer]
Now we focus on another part of the implementation: extracting and validating individuals' names from text using a combination of regular expressions and a generative AI model. This component is designed to handle multilingual and unstructured inputs effectively. The code begins by importing essential libraries—regex for advanced pattern matching with Unicode support, and transformers and torch to load the Mistral model (mistralai/Mistral-7B-Instruct-v0.3) and configure it for execution on GPU or CPU, with secure access via a Hugging Face authentication token.

The core logic includes a function, extract_sequences, which identifies personal name patterns such as initials, full names with titles, capitalized sequences, and names with linguistic prefixes. These matches are consolidated into a filtered list of potential names. Additional functions like clean and clean_name ensure consistent formatting by removing excess whitespace, handling uppercase sequences, and accounting for cultural name structures.

To validate whether a string is likely a personal name, the is_person_name function constructs a structured prompt for the Mistral model and generates low-randomness responses for consistency. The file processing workflow reads input data line by line, splits each line using the delimiter &&&****&&&, and extracts the segment to be processed. Regex is first applied to extract name candidates, which are then validated by the AI model and formatted accordingly.

The final output includes the company name, validated personal names, and the original raw input, written to a new file with _people.txt appended to the filename. This approach enables scalable, efficient, and language-agnostic name extraction while ensuring robustness against variations in formatting, abbreviations, and irrelevant text.

In [ ]:
```
import regex as re  # Importing regex module which supports Unicode property escape

from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
# Replace 'your_huggingface_token' with your actual Hugging Face access token
token = 'your_token'

# Load the model and tokenizer with the token for authentication
device = "cuda" if torch.cuda.is_available() else "cpu"

model_name = "mistralai/Mistral-7B-Instruct-v0.3"
tokenizer = AutoTokenizer.from_pretrained(model_name, token=token)
model = AutoModelForCausalLM.from_pretrained(model_name, token=token).to(device)

def extract_sequences(text):
    """
    Extract sequences of words that begin with a capital letter from the given text

    Args:
    text (str): The input text to search within.

    Returns:
    tuple: Lists for "A. Dickes" style sequences, multi-word capitalized sequences,
           names with geographical or familial prefixes, sequences with titles, and
    """
    # Remove extra whitespace from the text
    text = re.sub(r'\s+', ' ', text)

    # Existing patterns
    pattern_abbr = r'\b[A-ZÄÖÜßÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖØÙÚÛÜÝŸČĆŠŽŒ]\.\s+[A-ZÄÖÜßÀÁÂ

    pattern_full_name = r'\b(?:M\.|Mlle|Mme|Monsieur|Madame|Mademoiselle|Messieurs|

    pattern_multi_word = r'\b(?:[A-ZÄÖÜßÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖØÙÚÛÜÝŸČĆŠŽŒ][a-zäöü

    pattern_prefix = r'\b[A-ZÄÖÜßÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖØÙÚÛÜÝŸČĆŠŽŒ][a-zäöüßàáâãäå

    pattern_general = r'\b(?:[A-ZÄÖÜßÀÁÂÃÄÅÆÇÈÉÊËÌÍÎÏÐÑÒÓÔÕÖØÙÚÛÜÝŸČĆŠŽŒ][a-zäöüßàá

    # New pattern for names with initials and surname, such as "Monsieur H.J. Sulke
    pattern_initials_with_title = r'\b(?:M\.|Monsieur|Madame|Herr|Frau|Dr\.|Doktor|


    # Find all matches for "A. Dickes" style sequences
    matches_abbr = re.findall(pattern_abbr, text)

    # Find all matches for full names including those with initials
    matches_full_name = re.findall(pattern_full_name, text)

    # Find all matches for multi-word capitalized sequences
    matches_multi_word = re.findall(pattern_multi_word, text)

    # Find all matches for names with prefixes like "de", "van", "von", "de la", et
    matches_prefix = re.findall(pattern_prefix, text)

    # Find all matches for general capitalized word sequences
```

```
    matches_general = re.findall(pattern_general, text)

    # Find all matches for pattern_initials_with_title sequences
    matches_pattern_initials_with_title = re.findall(pattern_initials_with_title, t

    # Combine all matches into one list for multi-word capitalized sequences
    combined_matches_multi_word = set(matches_full_name + matches_multi_word + matc

    cr = []
    for each_item_c in combined_matches_multi_word:
        if len(each_item_c.split(" ")) > 1:
            cr.append(each_item_c)

    # Remove any matches that are already included in matches_abbr to avoid duplica
    combined_matches_multi_word = [match.strip() for match in cr if match.strip() n

    return matches_abbr, combined_matches_multi_word, matches_prefix, matches_full_


def clean(text_in):
    return re.sub(r'\s+', ' ', text_in).strip()

def clean_name(name):
    # Split the name into parts to handle the first word separately
    name = name.replace("MM ", "")
    parts = name.split()

    # Process only the first part to clean names
    first_part = parts[0]

    # Check if the first part has multiple consecutive uppercase letters and at lea
    index = 0
    for char in first_part:
        if char.isupper():
            index += 1
        else:
            break

    # If the first word has more than one consecutive uppercase letter and at least
    if index > 1 and any(char.islower() for char in first_part):
        first_part = first_part[index-1:]

    # Replace the first part in the list of parts
    parts[0] = first_part

    # Reassemble the cleaned parts into a single string
    cleaned_name = ' '.join(parts).strip()

    return cleaned_name

# Function to use the model to check if a text is a person's name
def is_person_name(name):
    prompt = (f"Determine if the following text is a person's name. "
              f"Consider potential typos or cultural variations in names: '{name}'.
              f"Additionally, phrases such as 'Recueil Spécial' or 'Le Receveur' or
              f"If the text is highly likely a person's name, answer 'yes'. Otherwi
```

```
    # Tokenize and move to correct device
    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    # Generate response on the same device
    outputs = model.generate(
        **inputs,
        max_new_tokens=30,
        temperature=0.2  # Lower randomness for consistent answers
    )

    response = tokenizer.decode(outputs[0], skip_special_tokens=True).replace("'yes

    if "Answer:" in response:
        answer = response.split("Answer:")[-1].strip().lower()
    else:
        answer = response.strip().lower()

    return 'yes' in answer


# Read the file line by line with Latin-1 encoding

pathfiles = [
    '/JDH_PAPER/extracted_company_names.txt'
]

for file_path in pathfiles:

    write = open(file_path.replace(".txt", "_people.txt"), "w", encoding='utf-8')

    with open(file_path, 'r', encoding='utf-8') as file:
        for line in file:
            # Step 1: Strip the line and split using the specified delimiter "&&&**
            parts = line.strip().replace(" MM"," MM ").split('&&&****&&&')
            company = clean(parts[0])
            print("company: ", company)
            # Step 2: Extract the last item from the split parts
            last_part = clean(parts[-1]).strip()
            if "wrong" not in company.lower():
                # Step 3: Call the function to extract sequences
                list_abbr, list_multi_word, list_prefix, list_full_name, list_gener

                # Step 4: Print the results for this line
                print("list_abbr: ", list_abbr)
                abb = ""
                for each_i in range(0, len(list_abbr) - 1):
                    list_abbr[each_i] = clean(list_abbr[each_i])
                    abb = abb + list_abbr[each_i] + " *_* "
                if len(list_abbr) > 0:
                    abb = abb + list_abbr[-1]
                print("abb: ", abb)

                print("list_multi_word: ", list_multi_word)
                filtered_names = [name for name in list_multi_word if is_person_nam
                print(filtered_names)
```

```
                filtered_names_1=[]
                for Fil in filtered_names:
                    if Fil not in  filtered_names_1:
                        filtered_names_1.append(Fil)
                filtered_names=filtered_names_1
                print("filtered_names: ", filtered_names)

                fn = ""
                for each_i in range(0, len(filtered_names) - 1):
                    filtered_names[each_i] = clean(filtered_names[each_i])
                    filtered_names[each_i] = clean_name(filtered_names[each_i])
                    fn = fn + filtered_names[each_i] + " *_* "
                if len(filtered_names) > 0:
                    fn = fn + filtered_names[-1]
            else:
                fn=""
                abb=""
            print("fn: ", fn)
            print("*****************************************************************

            fn = clean(fn)
            abb = clean(abb)

            print("******************")
            newline = company.replace('"',"") + " &&&***&&& " + company.replace('"'
            last_part + "\n"
            write.write(newline)
            write.flush()

            print("-" * 40)  # Separator for output clarity
    write.close()
print("done")
```

[Hermeneutics layer]

We next explore another key component of the overall implementation: a pipeline for fine-tuning a GPT-2 model with enhanced capabilities for efficient training, response generation, and model merging. This section complements the previous modules by introducing methods for adapting large language models to specific tasks using Low-Rank Adaptation (LoRA), managing system resources effectively, and integrating fine-tuned parameters into base models for flexible deployment. The process begins with class initialization, where core attributes such as the dataset path, cache directory, and Hugging Face authentication token are defined. A custom GPT-2 configuration is established by adjusting architecture parameters like hidden size, attention heads, number of layers, vocabulary size, and activation function. Using this configuration, the tokenizer and model are loaded from Hugging Face, and the dataset—assumed to be in JSON format—is imported and prepared using the datasets library. Fine-tuning is carried out using LoRA, which allows selective updating of specific layers in the model to reduce training overhead. Training parameters, such as epoch count, batch size, and logging intervals, are specified and passed to a supervised fine-tuning trainer, which handles the model training process. The fine-tuned model is saved to a designated output directory, with training metrics optionally tracked via

TensorBoard. For inference, the pipeline includes a response generation method that constructs a prompt, tokenizes it, and produces a concise answer using the fine-tuned model. Generation behavior is controlled through parameters like output length and temperature. The output is then decoded and cleaned to return a focused response. Resource management is addressed through a cleanup function that removes model components from memory and clears the GPU cache. Additionally, the code supports selective model merging, which involves loading both the base and fine-tuned models, comparing their parameters, and updating the base model with selected weights—ensuring compatibility before saving the merged model. This module fits into the larger architecture as a reusable and scalable fine-tuning pipeline that supports both training and inference. The following code block provides the full implementation of this component.

In [ ]:
```
import gc
import torch
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    GPT2Config  # Import the GPT-2 configuration class
)
from peft import LoraConfig, PeftModel
from random import randrange
from trl import SFTTrainer  # Correct import for the SFTTrainer

class FineTune_GPT2:
    def __init__(self, dataset_path, cache_dir, token):
        """
        Initialize the FineTune_GPT2 class.

        Args:
            dataset_path (str): Path to the dataset.
            cache_dir (str): Directory to cache the models and tokenizers.
            token (str): Hugging Face API token for accessing gated repositories.

        How to obtain the Hugging Face API token:
        1. Go to https://huggingface.co/settings/tokens
        2. Log in or sign up if you don't have an account.
        3. Create a new token with the required permissions (read access is suffici
        4. Copy the token and provide it when initializing this class.
        """
        self.dataset_path = dataset_path
        self.cache_dir = cache_dir
        self.token = token

        # Embedded configuration
        config_dict = {
            "architectures": ["GPT2LMHeadModel"],
            "bos_token_id": 50256,
            "eos_token_id": 50256,
            "hidden_act": "gelu_new",
            "hidden_size": 768,
```

```
            "initializer_range": 0.02,
            "intermediate_size": None,
            "layer_norm_eps": 1e-05,
            "model_type": "gpt2",
            "num_attention_heads": 12,
            "num_hidden_layers": 12,
            "vocab_size": 50257
        }
        self.config = GPT2Config.from_dict(config_dict)  # Use the correct configur

        self.tokenizer = AutoTokenizer.from_pretrained(
            "gpt2",
            cache_dir=self.cache_dir,
            use_auth_token=self.token  # Use the correct parameter
        )
        self.tokenizer.pad_token = self.tokenizer.eos_token  # Set padding token to
        self.tokenizer.padding_side = 'right'  # Ensure padding is on the right
        self.trained_model = AutoModelForCausalLM.from_pretrained(
            "gpt2",
            config=self.config,
            cache_dir=self.cache_dir,
            use_auth_token=self.token  # Use the correct parameter
        ).to("cuda")
        self.dataset = load_dataset('json', data_files=self.dataset_path, split='tr

    def train_model(self, output_dir, num_train_epochs=3, per_device_train_batch_si
        lora_config = LoraConfig(
            lora_alpha=16,
            lora_dropout=0.1,
            r=64,
            target_modules=[
                "attn.c_attn",
                "attn.c_proj",
                "mlp.c_fc",
                "mlp.c_proj",
            ],
            bias="none",
            task_type="CAUSAL_LM",
        )

        training_args = TrainingArguments(
            output_dir=output_dir,
            num_train_epochs=num_train_epochs,
            per_device_train_batch_size=per_device_train_batch_size,
            per_device_eval_batch_size=per_device_eval_batch_size,
            logging_dir=f"{output_dir}/logs",
            logging_steps=10000,
            save_steps=1000,
            warmup_ratio=0.03,
            report_to="tensorboard"
        )

        trainer = SFTTrainer(
            model=self.trained_model,
            train_dataset=self.dataset,
            peft_config=lora_config,
```

```
            dataset_text_field="text",  # Assuming 'text' is the field name contain
            max_seq_length=max_seq_length,  # Pass None or specify a maximum sequen
            tokenizer=self.tokenizer,
            args=training_args
        )

        trainer.train()
        trainer.model.save_pretrained(output_dir)
        print("The new model is available in " + output_dir)
        self.trained_model = trainer.model

    def generate_response(self, question, max_new_tokens=500, temperature=0.1):
        prompt = f"""You will be provided with a question. You must provide only a
        Question:
        {question}
        """
        model_input = self.tokenizer(prompt, return_tensors="pt").to("cuda")
        with torch.no_grad():
            generated_code = self.trained_model.generate(**model_input, max_new_tok
            generated_code = self.tokenizer.decode(generated_code[0], skip_special_
            response = generated_code.split("You will be provided with a question")
            if len(response) < 10:
                return generated_code
        return response

    def clean_up(self):
        del self.tokenizer
        del self.trained_model
        gc.collect()
        torch.cuda.empty_cache()

    def selective_merge(self, base_model_path, fine_tuned_model_path, output_dir):
        base_model = AutoModelForCausalLM.from_pretrained(base_model_path, cache_di
        fine_tuned_model = AutoModelForCausalLM.from_pretrained(fine_tuned_model_pa

        # Extract state dicts
        base_state_dict = base_model.state_dict()
        ft_state_dict = fine_tuned_model.state_dict()

        # Filter out keys: only update base model with keys that exist in its state
        for key in ft_state_dict:
            if key in base_state_dict and ft_state_dict[key].size() == base_state_d
                base_state_dict[key] = ft_state_dict[key]

        # Load the filtered state dict back into the base model
        base_model.load_state_dict(base_state_dict, strict=False)

        # Save the merged model
        base_model.save_pretrained(output_dir)

        # Save tokenizer and configuration as well
        self.tokenizer.save_pretrained(output_dir)
        self.config.save_pretrained(output_dir)

        print(f"Merged model, tokenizer, and config are saved in {output_dir}")
```

```
        return base_model



# Instantiate and use the FineTune_GPT2 class
dataset_path = "/JDH_PAPER/training.jsonl"
hf_token = "............." # your token
cache_dir = "/JDH_PAPER"
output_dir = "/JDH_PAPER/Models/"
merged_model_output_dir = "/JDH_PAPER/Merged"

# Create an instance of FineTune_GPT2 and start training
fine_tuner = FineTune_GPT2(dataset_path, cache_dir, token=hf_token)


# Train the model and save it to the output directory
fine_tuner.train_model(output_dir=output_dir, num_train_epochs=25, per_device_train

# Optionally, merge the fine-tuned model with the base model
fine_tuner.selective_merge(base_model_path=output_dir, fine_tuned_model_path=output
```

[Hermeneutics layer]
Building on the previous components, this section demonstrates how a fine-tuned GPT-2 model is used to extract addresses from unstructured text through a complete pipeline of preprocessing, inference, and post-processing. The process begins with initializing a class that loads the tokenizer and model from a specified directory, using a Hugging Face token and memory-efficient settings such as half-precision (torch.float16) and right-side padding for token alignment. To guide the model's output, the code constructs a question-style prompt, which is tokenized and passed through the model using controlled generation parameters like maximum token length and temperature for deterministic responses. The output is then decoded and cleaned to remove special tokens or redundant text. Input is read from a structured file where each line contains company data. Each line is split using a custom delimiter, and the relevant portion is extracted to form the prompt. The generated response is then processed using regex to isolate the address, clean it, and apply fallback logic in cases where a valid address cannot be determined. Following address extraction, the output undergoes an additional cleaning stage to enhance readability, and results are written to a new file with consistent formatting. The code includes error-handling procedures that reinitialize the model if failures occur and employs memory management techniques such as garbage collection and GPU cache clearing. This module integrates seamlessly into the larger system, handling input, generating and refining address data, and producing clean, structured output ready for further analysis.

In [ ]:
```
from transformers import AutoModelForCausalLM, AutoTokenizer
import gc
import torch
import os,re


class FineTune_GPT2_use:
```

```
    def __init__(self, cache_dir=None, token=None, model_path=None):
        """
        Initialize the class with cache_dir, token, and model_path for the GPT2 mod
        """
        self.cache_dir = cache_dir
        self.token = token

        # Load the tokenizer and model from the specified path (fine-tuned model fo
        self.tokenizer = AutoTokenizer.from_pretrained(model_path, cache_dir=self.c

        # Load the model on CPU for debugging, or on CUDA with float16 if memory is
        self.trained_model = AutoModelForCausalLM.from_pretrained(
            model_path,
            cache_dir=self.cache_dir,
            token=self.token,
            torch_dtype=torch.float16  # Try using float16 to reduce memory
        ).to("cuda")

        # Set padding token for the tokenizer
        # self.tokenizer.pad_token = self.tokenizer.eos_token
        self.tokenizer.padding_side = 'right'

    def generate_response(self, question, max_new_tokens=500, temperature=0.1):
        """
        Generate a response to the given question using the fine-tuned GPT2 model.

        Args:
            question (str): The question to ask the model.
            max_new_tokens (int): Maximum number of tokens to generate.
            temperature (float): The temperature for generation.

        Returns:
            str: The generated response.
        """
        device = "cuda" if torch.cuda.is_available() else "cpu"

        # Tokenize the input and print for debugging
        inputs = self.tokenizer(question, return_tensors="pt", truncation=True, max
        inputs = inputs.to(device)  # Move inputs to the same device as the model

        # Generate text with no gradient computation
        with torch.no_grad():
            generated_output = self.trained_model.generate(
                **inputs,
                max_new_tokens=max_new_tokens,
                pad_token_id=self.tokenizer.eos_token_id,
                temperature=temperature
            )
            # Decode the generated text and remove special tokens
            generated_text = self.tokenizer.decode(generated_output[0], skip_specia

        return generated_text

    def clean_up(self):
        """
        Clean up the resources by deleting the model and tokenizer, and clearing th
```

```
        """
        del self.tokenizer
        del self.trained_model
        gc.collect()
        torch.cuda.empty_cache()

# Run the model on CPU or GPU with debug options to find the problem
if __name__ == "__main__":
    gpt2_model_path = "/JDH_PAPER/Merged/"
    fine_tuner1 = FineTune_GPT2_use(cache_dir='/JDH_PAPER', token='use_your_token',
    pathfiles = [
        '/JDH_PAPER/extracted_company_names_people.txt'
    ]
#     question = "Where is the address of the company in the following text?"
#     response = fine_tuner.generate_response(question)
#     print("response:",response)
    for each_fine in pathfiles:
        dataset_path = each_fine.replace("_people.txt", "_people_Addr.txt")
        write = open(dataset_path, "w", encoding='utf-8')
        read_file = each_fine

        read = open(read_file, 'r', encoding='utf-8')
        for line in read:
            # Process each line and extract relevant information
            L = line.split(" &&&***&&& ")
            L[-1]=L[-1].rstrip().lstrip().strip()
            CopyL=L[-1]
            L[-1] = L[-1].split("+++++++++++++++++")[-1].replace(" : ", ": ")

            # Step 1: Use GPT-2 fine-tuned model to find address in text
            te=""
            size_L=50
            if len(L[-1].split(" "))<50:
                size_L=len(L[-1].split(" "))
            for e in range(0,size_L):
                te=te+L[-1].split(" ")[e]+" "
            question="Where is the address of the company in the following text? "+
            print("question: ",question)
            try:
                address=fine_tuner1.generate_response(question)
            except:
                del fine_tuner1
                fine_tuner1 = FineTune_GPT2_use(cache_dir='/JDH_PAPER/', token='you

            address = re.sub(r'\s+', ' ', address)  # Clean the response

            try:
                # Extract address, handle missing or incorrect data gracefully
                address = re.sub(r'\s+', ' ', address).split("Answer:")[1].replace(
            except:
                print("To be checked: ", address)
                address = "No Addr"

            address = re.sub(r'\s+', ' ', address)
            address=re.sub(r"(?<!^)(?=[A-Z])", " ", address)
            address = re.sub(r'\s+', ' ', address)  # Clean the response
```

```
            if len(address)>120:
                address="No Addr"

            refined_address = address  # Refinement step could be added here
            print("++++++++++")
            print("refined address:", refined_address)
            print("++++++++++")
            print("==================================================================

            # Write results back to the file
            newline = L[0]+" &&&***&&& "+L[1]+" &&&***&&& "\
            +L[2]+" &&&***&&& "+L[3]+" &&&***&&& "+refined_address+" &&&***&&& "+Co

            write.write(newline)
            write.flush()

        write.close()
        read.close()

        # Clean up resources after processing
        fine_tuner1.clean_up()
```

[Hermeneutics layer]
Expanding the capabilities of the pipeline, this module focuses on identifying and extracting country names from text using a multilingual dictionary and regex-based matching. A predefined dictionary maps country names and their variations in English, French, German, and Luxembourgish. The code reads an input file containing company information, splits each line into structured components using a custom delimiter, and applies text cleaning to normalize whitespace. For each line, it scans the relevant text component—typically a description or address—against all known country name variations. A for loop is used to match standalone country names while avoiding partial matches, with special handling to distinguish cases like “Jersey” and “New Jersey” based on frequency. Extracted country names are deduplicated and formatted using a custom separator before being appended to the original data. The processed results are written to a new output file in a structured format, ensuring compatibility with downstream tasks. Throughout execution, the code prints intermediate values such as company names, matched text, and identified countries for verification. This module adds robust, multilingual country name extraction to the processing pipeline, with careful handling of edge cases and clear formatting for analysis.

In [ ]:
```
file="/JDH_PAPER/extracted_company_names_people_Addr.txt"
read=open(file,"r",encoding="utf-8")
write=open("/JDH_PAPER/final_a.txt","w",encoding="utf-8")
import re
for line in read:
    L=line.split("&&&***&&&")
    names_c=[]
    for i in range(0, len(L)):
        L[i] = re.sub(r'\s+', ' ', L[i]).strip()
    string_name_co=" "
    if "wrong" in L[0].lower():
```

```
        pass
    else:
        found = set()
        for english_name, variants in countries.items():
            all_variants = set(variant.lower() for variant in variants + [english_n

            if english_name == "Jersey":
                if is_clean_match(normalized_text, "jersey") and not is_clean_match
                    found.add("Jersey")
                    JJCC += 1
            else:
                for variant in all_variants:
                    if is_clean_match(normalized_text, variant):
                        found.add(english_name)

        names_c=list(found)
        for e in names_c:
            string_name_co=string_name_co+e+" *_* "
        string_name_co=re.sub(r'\s+', ' ', string_name_co).strip()[:-4]
    newline=""
    for i in range(0,len(L)-1):
       newline=newline+L[i]+" &&&***&&& "
    newline=newline+string_name_co+" &&&***&&& " +L[-1]
    newline=re.sub(r'\s+', ' ', newline).strip()
    print("Company: \n ",L[0])
    print("Description: \n ",L[-1].split("+++++++++++++++++")[-1])
    print("++++++++")
    print("Countries:")
    print(string_name_co)
    print("++++++++")

    print("*************************************************************************

    write.write(newline.replace("\n","")+"\n")
    write.flush()
write.close()
print("done")
```

[Hermeneutics layer]

As part of the people extraction pipeline, an additional improvement step is introduced to enhance data consistency by removing formal titles from extracted person names. This refinement ensures that names are cleaned before entering the loading phase, eliminating job roles, honorifics, and linguistic variations that can introduce noise in downstream analysis. The cleaning process begins with a function to remove excess whitespace, using regular expressions to replace multiple spaces with a single space and trimming any leading or trailing characters. The core of the enhancement lies in the title removal function, which relies on a comprehensive, multilingual list of titles, including English, French, German, Luxembourgish, Spanish, and Portuguese variants. A flexible regex pattern is constructed to match these titles as standalone words, accounting for optional punctuation and accented characters. Matches are removed in a case-insensitive and Unicode-aware manner to ensure robustness across languages. Once processed, the names are returned in a clean, title-free format. For example, inputs such as “Dr. John Doe,” “Monsieur Jean Dupont,” or “Herr Karl

Müller" are standardised to "John Doe," "Jean Dupont," and "Karl Müller," respectively. This step significantly improves the uniformity and accuracy of person name data, simplifies further processing, and adapts easily to multilingual contexts. Integrated directly before the loading stage, this module ensures that only properly cleaned names are included in the final dataset, contributing to overall data quality and reliability within the pipeline.

In [ ]:
```
def remove_extra_spaces(text):
    # Use regular expression to replace multiple spaces with a single space
    cleaned_text = re.sub(r'\s+', ' ', text)
    # Strip leading and trailing spaces
    cleaned_text = cleaned_text.strip()
    return cleaned_text

def remove_titles(name):
    import re

    # List of common titles in English, French, German, Luxembourgish, Spanish, and
    titles = [
        # English Titles
        r'\bMr\b', r'\bMrs\b', r'\bMs\b', r'\bMiss\b', r'\bDr\b', r'\bProf\b', r'\b
        r'\bMadam\b', r'\bLord\b', r'\bLady\b', r'\bRev\b', r'\bHon\b', r'\bJudge\b
        r'\bDame\b', r'\bCapt\b', r'\bCol\b', r'\bGen\b', r'\bLt\b', r'\bMaj\b',
        r'\bSgt\b', r'\bCpl\b', r'\bPvt\b', r'\bChief\b', r'\bOfficer\b', r'\bDetec
        r'\bAttorney\b', r'\bAmb\b', r'\bConsul\b', r'\bSec\b', r'\bDir\b', r'\bMgr
        r'\bCEO\b', r'\bCFO\b', r'\bCTO\b', r'\bPresident\b', r'\bVP\b', r'\bDep\b'

        # French Titles
        r'\bMonsieur\b', r'\bMadame\b', r'\bMademoiselle\b', r'\bMlle\b', r'\bMme\b
        r'\bDocteur\b', r'\bProfesseur\b', r'\bPrésident\b', r'\bVice-président\b',
        r'\bDirecteur\b', r'\bPDG\b', r'\bAdministrateur\b', r'\bComte\b', r'\bComt
        r'\bMarquis\b', r'\bMarquise\b', r'\bDuc\b', r'\bDuchesse\b', r'\bPrince\b'
        r'\bMister\b', r'\bMessieurs\b', # Mister with an ï (for completeness)

        # German Titles
        r'\bHerr\b', r'\bFrau\b', r'\bFräulein\b', r'\bHr\b', r'\bFr\b', r'\bDoktor
        r'\bProfessor\b', r'\bPräsident\b', r'\bVizepräsident\b', r'\bSekretär\b',
        r'\bDirektor\b', r'\bVerwalter\b', r'\bKanzler\b', r'\bBaron\b', r'\bBaroni
        r'\bGraf\b', r'\bGräfin\b', r'\bFürst\b', r'\bFürstin\b', r'\bHerzog\b', r'

        # Luxembourgish Titles
        r'\bHär\b', r'\bMadame\b', r'\bMademoiselle\b', r'\bDokter\b', r'\bProfesse
        r'\bBuergermeeschter\b', r'\bPrinz\b', r'\bPrinzessin\b', r'\bGroßherzog\b'

        # Spanish Titles
        r'\bSeñor\b', r'\bSeñora\b', r'\bSeñorita\b', r'\bDr\b', r'\bDra\b', r'\bPr
        r'\bDon\b', r'\bDoña\b', r'\bLicenciado\b', r'\bLicenciada\b', r'\bIngenier
        r'\bIngeniera\b', r'\bPresidente\b', r'\bVicepresidente\b', r'\bDirector\b'
        r'\bAdministradora\b', r'\bAlcalde\b', r'\bAlcaldesa\b',

        # Portuguese Titles
        r'\bSenhor\b', r'\bSenhora\b', r'\bSenhorita\b', r'\bDoutor\b', r'\bDoutora
        r'\bProfessor\b', r'\bProfessora\b', r'\bEngenheiro\b', r'\bEngenheira\b',
        r'\bPresidente\b', r'\bVice-presidente\b', r'\bDiretor\b', r'\bDiretora\b',
        r'\bAdministrador\b', r'\bAdministradora\b', r'\bPrefeito\b', r'\bPrefeita\
```

```
    ]

    # Create a regex pattern that matches any of the titles (including accented cha
    pattern = re.compile(r'\b(?:' + '|'.join(titles) + r')\.?', re.IGNORECASE | re.

    # Substitute the titles with an empty string
    name_without_titles = re.sub(pattern, '', name).strip()
    return name_without_titles
import re
read=open("/JDH_PAPER/final_a.txt","r",encoding="utf-8")
write=open("/JDH_PAPER/final_a.txt".replace("_a.txt","_c.txt"),"w",encoding="utf-8"

delimiter="&&&***&&&"
count=0
for line in read:
    L=line.split(delimiter)
    L[2]=remove_extra_spaces(L[2]).rstrip().lstrip().strip()
    print("Input names: ",L[2])
    print()
    newL2=L[2].split(" *_* ")

    PEOPLE=[]
    output_name=[]
    for index in range(0,len(newL2)):
        newL2[index]=remove_titles(remove_extra_spaces(newL2[index]).lstrip().rstri
        if len(newL2[index].split(" "))>1:
            if len(newL2[index].split(" ")[-1])>2:
                output_name.append(remove_extra_spaces(newL2[index]).lstrip().rstri
    print("Cleaned names: ",output_name)
    newline=""
    for i in range(0,len(L)-1):
        if i!=2:
            newline=newline+L[i]+" "+delimiter+" "

        else:
            output_string=""
            for each_n in output_name:
                output_string=output_string+each_n+" *_* "
            output_string=output_string[:-4].lstrip().rstrip().strip()
            print("output_string:",output_string)
            newline=newline+output_string+" "+delimiter+" "
    newline=newline+L[-1]
    write.write(newline)
    write.flush()
    print()
    print("**************************************************")
read.close()
write.close()
```

Load

[Hermeneutics layer]

To complete the ETL pipeline, the final step involves loading the processed and cleaned data into a relational database, ensuring both data integrity and the establishment of meaningful

relationships between entities such as companies, addresses, individuals, countries, and messages. This stage begins by connecting to a SQL Server database using pyodbc, with secure authentication and error handling to validate connectivity. For efficiency, the implementation preloads existing records into in-memory dictionaries, reducing redundant queries by caching entity IDs. Before insertion, values are standardized using cleaning functions to ensure consistency. Helper methods manage the insertion or retrieval of primary keys by first checking for existing entries and inserting new ones only when necessary. These keys are then used to establish associations across multiple junction tables, supporting complex many-to-many relationships. The pipeline processes an input file line by line, splitting each record into structured components using a custom delimiter. Company names, alternate names, addresses, person names, and countries are extracted, cleaned, and matched against the database. When a match is found or inserted, corresponding IDs are retrieved and used to link data across the relational schema. Descriptions are recorded in a messages table and connected to relevant entities via junction tables. Special handling is included to manage address length limits and ambiguous company names.The code dynamically adjusts to different file encodings and logs errors during any part of the process—such as file reading, database operations, or line parsing—ensuring that individual failures do not interrupt the overall workflow. Upon successful processing, the data is committed to the database, and all resources are properly released.By preventing duplicates, maintaining clean formatting, and efficiently linking entities, this step finalizes the ETL pipeline with a scalable and resilient approach to structured data integration, enabling robust querying and downstream analytical tasks.

In [ ]:
```
import pyodbc

# Define your connection string with the correct credentials and server information
conn_str = 'DRIVER={ODBC Driver 17 for SQL Server};SERVER=10.184.4.29;DATABASE=LETT

# Establish the connection
try:
    conn = pyodbc.connect(conn_str)
    cursor = conn.cursor()
    print("Connection successful")
except pyodbc.Error as e:
    print(f"Error connecting to the database: {e}")
    raise

# List of tables to delete from
# Note: Order matters - delete from the junction tables first to avoid foreign key
tables = [
    'People_Addresses',
    'People_Companies',
    'People_Countries',
    'People_MessagesL',
    'Addresses_Countries',
    'Companies_Countries',
    'People',
    'Addresses',
    'Companies',
```

```
    'Countries',
    'MessagesL',
]

# Function to delete all rows from the tables
def delete_all_data():
    try:
        for table in tables:
            print(f"Deleting data from {table}...")
            cursor.execute(f"DELETE FROM {table}")  # Using DELETE to respect forei
            conn.commit()
            print(f"Data deleted from {table}")
    except pyodbc.Error as e:
        print(f"Error deleting data from {table}: {e}")
        raise

# Run the delete process
delete_all_data()

# Close the connection
cursor.close()
conn.close()
print("Connection closed")
import pyodbc
import re
import traceback


def has_column(table, column):
    """
    Check if a column exists in a specific table.
    """
    try:
        cursor.execute("""
            SELECT 1
            FROM INFORMATION_SCHEMA.COLUMNS
            WHERE TABLE_NAME = ? AND COLUMN_NAME = ?
        """, table, column)
        return cursor.fetchone() is not None
    except pyodbc.Error as e:
        print(f"Error checking column {column} in table {table}: {e}")
        return False


def assign_unused_id(table, id_column):
    """
    Retrieve the next unused ID for a given table and column.
    """
    try:
        cursor.execute(f"SELECT MAX({id_column}) FROM {table}")
        max_id = cursor.fetchone()[0]
        return (max_id or 0) + 1
    except pyodbc.Error as e:
        print(f"Error fetching max ID from {table}: {e}")
        raise
```

```python
def insert_or_get_id(table, column, value, year=None, id_column=None):
    """
    Insert a new record or retrieve the ID of an existing record.
    """

    if not id_column:
        id_column = f"{table[:-1]}_id"

    try:
        # Check if the record already exists
        query = f"SELECT {id_column} FROM {table} WHERE {column} = ?"
        params = [value]
        if has_column(table, "year"):
            query += " AND year = ?"
            params.append(year)

        cursor.execute(query, params)
        row = cursor.fetchone()

        # Return the ID if the record exists
        if row:
            return row[0]

        # Assign new ID
        new_id = assign_unused_id(table, id_column)

        # Insert new record
        columns = [id_column, column]
        values = [new_id, value]
        placeholders = ["?", "?"]

        if has_column(table, "year") and year is not None:
            columns.append("year")
            values.append(year)
            placeholders.append("?")

        if has_column(table, "effective_start_date"):
            columns.append("effective_start_date")
            placeholders.append("?")
            values.append("1970-01-01")  # Default start date

        if has_column(table, "is_current"):
            columns.append("is_current")
            placeholders.append("?")
            values.append(1)  # Default value for is_current

        if has_column(table, "current_version"):
            columns.append("current_version")
            placeholders.append("?")
            values.append(1)  # Default version for new records

        insert_query = f"INSERT INTO {table} ({', '.join(columns)}) VALUES ({', '.j
        cursor.execute(insert_query, values)
        conn.commit()
```

```
        return new_id

    except pyodbc.Error as e:
        print(f"Error inserting or retrieving ID from {table} for value '{value}':
        return None


def insert_into_junction(table, col1, col2, year, id1, id2):
    """
    Insert or update a relationship in a junction table using SCD Type 2 logic.
    """
    try:
        # Check if an active record exists
        cursor.execute(f"""
            SELECT is_current, effective_start_date, effective_end_date
            FROM {table}
            WHERE {col1} = ? AND {col2} = ? AND year = ?
        """, id1, id2, year)
        row = cursor.fetchone()

        if row and row[0] == 1:  # Active record exists
            return  # No need to update

        if row:  # Inactivate the existing record
            cursor.execute(f"""
                UPDATE {table}
                SET is_current = 0, effective_end_date = GETDATE()
                WHERE {col1} = ? AND {col2} = ? AND year = ? AND is_current = 1
            """, id1, id2, year)
            conn.commit()

        # Insert the new record
        cursor.execute(f"""
            INSERT INTO {table} ({col1}, {col2}, year, is_current, effective_start_
            VALUES (?, ?, ?, 1, GETDATE(), NULL)
        """, id1, id2, year)
        conn.commit()
    except pyodbc.Error as e:
        print(f"Error inserting into {table} (col1={col1}, col2={col2}, year={year}
        raise



def process_file(path_file):
    """
    Process the input file and populate the database with its data.
    """
    try:
        line_number=0
        with open(path_file, 'r', encoding='utf-8') as file:
            for line in file:
                # Split the line into columns
                columns = line.strip().split(' &&&***&&& ')
                # Extract the data from the line
                year = int(columns[0].strip())
                company_name = columns[1]
```

```
                address = columns[3]

                country_name = columns[7].split("*_*")
                for index_country in range(0,len(country_name)):
                    country_name[index_country]=country_name[index_country].rstrip(

                person_name = columns[5].split("*_*")
                for person_index in range(0,len(person_name)):
                    person_name[person_index]=person_name[person_index].rstrip().ls

                try:
                    person_name.remove('')
                except:
                    pass

                try:
                    person_name.remove('')
                except:
                    pass

                if "wrong" in company_name.lower():
                    continue
                else:
                # Insert or retrieve the IDs
                    country_ids=[]
                    person_ids=[]

                    company_id = insert_or_get_id('Companies', 'company_name', comp
                    address_id = insert_or_get_id('Addresses', 'address_description

                    for country in country_name:
                        country_id = insert_or_get_id('Countries', 'country_name',
                        country_ids.append(country_id)

                    for person in person_name:
                        person_id = insert_or_get_id('People', 'person_name', perso
                        person_ids.append(person_id)

                    # Insert into junction tables
                    for country_id in country_ids:
                        if company_id and country_id:
                            print("company_id:",company_id)
                            print("country_id:",country_id)
                            insert_into_junction('Companies_Countries', 'company_id
                        if address_id and country_id:
                            insert_into_junction('Addresses_Countries', 'address_id

                    for person_id in person_ids:
                        if person_id and company_id:
                            insert_into_junction('People_Companies', 'person_id', '
                        if person_id and address_id:
                            insert_into_junction('People_Addresses', 'person_id', '


                    print("line_number:",line_number)
                line_number=line_number+1
```

```
    except FileNotFoundError:
        print(f"File not found: {path_file}")
    except pyodbc.Error as e:
        print(f"Database error while processing file: {e}")
    except Exception as e:
        print(f"Unexpected error: {e}")


if __name__ == "__main__":
    conn_str = 'DRIVER={ODBC Driver 17 for SQL Server};SERVER=10.184.4.29;DATABASE=

    for year in range(1970, 1971):
        try:
            conn = pyodbc.connect(conn_str)
            cursor = conn.cursor()
            print(f"Processing data for year {year}...")
            path_file = f"../FILE_IMPORT/{year}_final_2ADDR_final_check_c.txt"
            process_file(path_file)
        except Exception as e:
            print(f"Error: {e}")
        finally:
            try:
                cursor.close()
                conn.close()
            except Exception:
                pass
```

Hyperlocalized geographies: shell companies in Luxembourg

[Narrative layer]

Comment Area:
[Demi: should we split the periods here as 1929-1960 and 1961-2016, instead of 1929-1959 and 1960-2016? We need to check this with the data]
[09/09/2025 - Benoît will formulate footnotes to make it explicit where the data is coming from]
[09/09/2025 - Benoît will check the periods for consistency]

During his first campaign to become president of the United States in 2009, Barack Obama made reference to the Ugland House in Georgetown, Cayman Islands, in several of his speeches. Qualified by the future American president as "either the biggest building in the world or the biggest tax scam in the world" (Davis 2009), the headquarter of the law firm Maples Group (previously Maples and Calder) is the siege of numerous companies: if the number of companies that are supposelly being hosted varies greatly in the press - from 18 000 to 40 000, it is clear that the siege of this service provider in the Cayman Islands that was also present in Luxembourg first as a financial service provider (specialized fiduciary services, fund administration, entity formation and management, insurance management...) since

2007 and then as a law firm since 2018, did host thousands of companies ('Le groupe Maples se développe au Grand-Duché' 2018). Most of the offshore financial centers have their "Ugland Houses" and so does Luxembourg. In order to map this phenomenon and to follow it over time, we proceeded in two ways. For the years 1929-1959, we built a database of holdings. For the years 1960-2016, a different method to identify the most relevant addresses for shell companies was used (cf. below

Shell companies in the center of the financial sector (1929-1960)

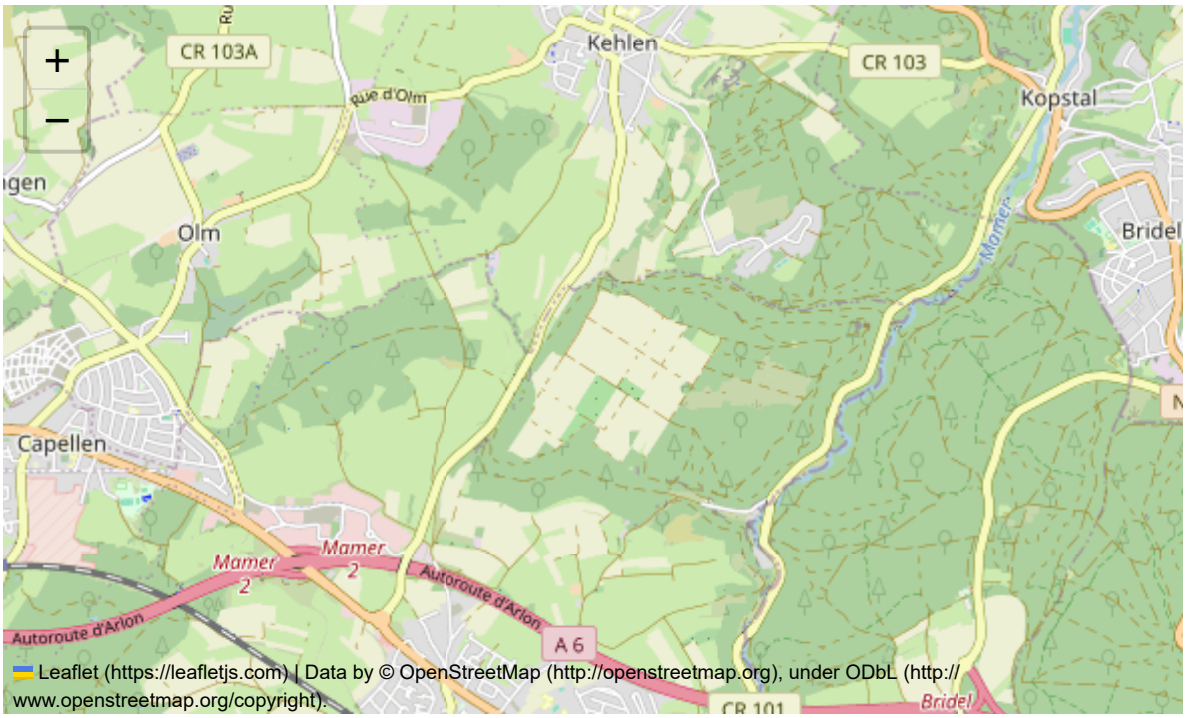
[Narrative layer]
For the years 1929 to 1960, we could identify 2791 holdings, 1373 of which with an exact address. Figure 3 presents how such addresses were distributed from 1929 to 1939 (Fig. 3A), and from 1945 to 1960 (Fig. 3B).

In [4]:
```
import folium
from folium.plugins import HeatMap
import pandas as pd

df = pd.read_csv('addresses01.csv', delimiter=',')

addresses = list(df['nom de la rue'])
firstHalf = list(df['1929-1939'])
secondHalf = list(df['1945-1959'])
latitudes = list(df['latitude'])
longitudes = list(df['longitude'])

geoFirst = []
geoSecond = []
for i in range(len(latitudes)):
    if firstHalf[i] != 0:
        for j in range(firstHalf[i]):
            geoFirst.append([latitudes[i], longitudes[i]])
    if secondHalf[i] != 0:
        for k in range(secondHalf[i]):
            geoSecond.append([latitudes[i], longitudes[i]])

# Create maps centered on Luxembourg City
m1 = folium.Map(
    location=[49.6116, 6.1319],  # Luxembourg City coordinates
    zoom_start=12.5,  # Adjust zoom level (higher = more zoomed in)
    tiles="OpenStreetMap"  # Default map style (can be changed)
)

m2 = folium.Map(
    location=[49.6116, 6.1319],  # Luxembourg City coordinates
    zoom_start=12.5,  # Adjust zoom level (higher = more zoomed in)
    tiles="OpenStreetMap"  # Default map style (can be changed)
)

HeatMap(geoFirst).add_to(m1)
```

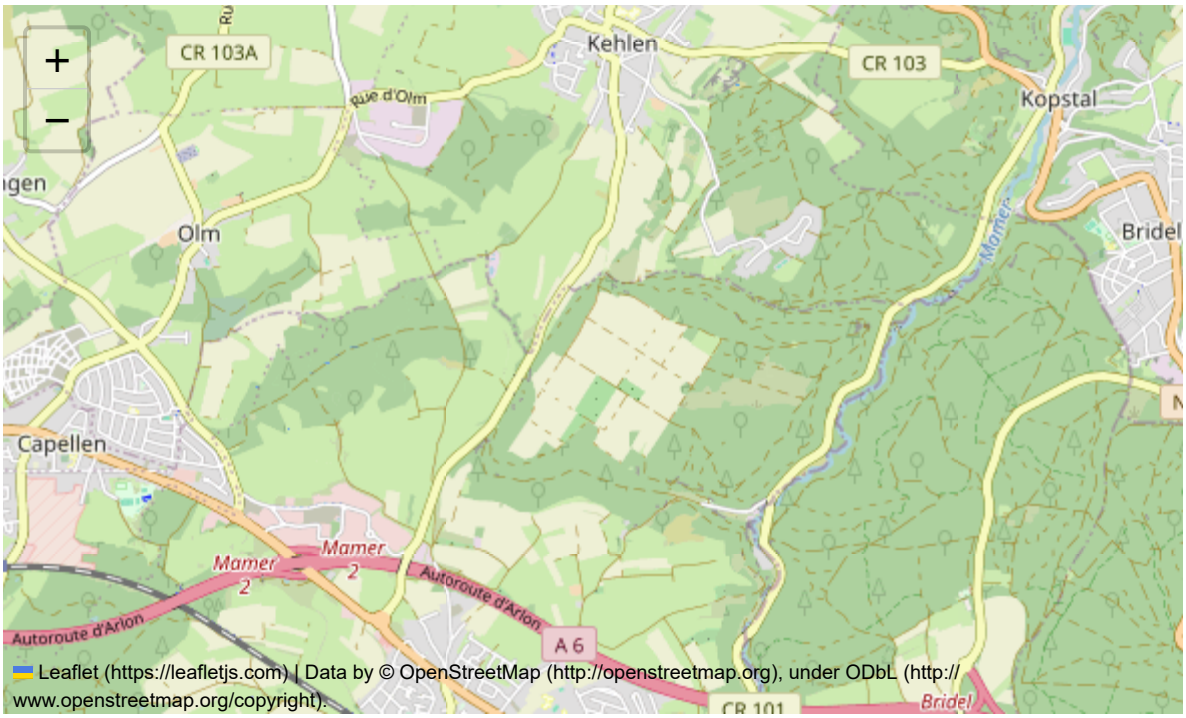
```
HeatMap(geoSecond).add_to(m2)
```

Out[4]: <folium.plugins.heat_map.HeatMap at 0x1d2c6fe0590>

In [5]:
```
display(m1)
```



In [6]:
```
display(m2)
```



[Figure] **Fig. 3.**

[Hermeneutics layer] We can highlight three elements from Figure 3. First, the financial center that emerged between the world wars is hyper localized. The vast majority of holdings

have their headquarters within a one-kilometer perimeter, concentrated around Boulevard Royal, which was constructed in the 1870s following the dismantling of the fortress. Luxembourg city was a small town of around 50,000 inhabitants. Several banks settled on this prestigious avenue, which mixes private villas and the first financial institutions at the beginning of the 20th century. The political, cultural and economic was geographically extremely concentrated (Philippart 2006a; 2006b; Weber 2013): it is there that the emerging holding market took off.

[Hermeneutics layer]

Comment Area:

[Demi: we need periods consistency here too]

Secondly, when linking addresses to companies, it appears that the market of holding domiciliation is dominated in the first years before the war by banks. Among the five most used addresses between 1929 and 1939, four are the headquarters of banks. Banking has been a shaky business in Luxembourg in the second half of the 19th century with several bankruptcies and the few banks remaining relatively small (Anders 1928). After World War One, French and Belgium capital invested in the Luxembourgish banking market so that at the end of the 1920s when the holding law passed parliament, the Grand Duchy had about fifteen banking institutions. Four of them invested the holding market: the International Bank of Luxembourg (BIL), which hold 42% of the holding company headquarters for which the address is known, the Commercial Bank 19%, the Levy Bank 14%, and Banque Générale du Luxembourg (BGL), which only entered the holding market in 1936, with 6%. Apart from BIL, which was founded in 1856, the other three banks had been established after World War One: BGL in 1919, the Commercial Bank in 1921, and Alfred Levy Bank in 1926. Among the five most used addresses that served as headquarters for the holdings is also that of a notary, Paul Kuborn. Alongside banks, notaries played an important role as a financial service provider in Luxembourg as they offered several banking services, especially for private individual clients but several notaries also played an important role in the nascent emerging shell company market. Five of the 32 notaries authorized to practice during the 1930, five are specialized in drafting holding company deeds, a lucrative business at a moment where the notarial profession was going through a serious crisis: with several firms were being liquidated because they had engaged in disastrous financial affairs (N. Majerus 1949, 2:822). Among the five notaries that were present on this market, Paul Kuborn was the most prolific: between 1929 and 1939, nearly a quarter of the holdings are documented in his office. Close to BIL, for which he managed most of the holding formations, he also provided the whole range of domiciliation services (address, sham directors...). While other notaries sometimes provided their clerks as sham directors—since they themselves were prohibited from performing such functions— they rarely offered their office addresses as holding headquarters.

[Hermeneutics layer]

Thirdly, World War Two did not constitute a real break in terms of the geographical location of the company domiciliation market. A number of holdings decided to leave Luxembourg in 1939 due to an increasingly uncertain political situation. However, the German occupiers did not interfere with the holding regime, neither to use it for their own needs nor to abolish it. Yet, interventions in the financial system by the occupiers were not without consequences. Out of the 16 banks active in Luxembourg at the time of the German invasion, most had to cease their operations. Alfred Levy et Compagnie Bank began to dissolve by the late 1930s: the occupiers could only seize the bank's headquarters and furniture, as all assets had been moved abroad: the main partner, Alfred Levy, considered Jewish by the Nazi occupiers, had left Luxembourg for the United States. The bank, which had been a major player in domiciliation, would not return to the Luxembourg market after 1944. Another bank, also defined as Jewish by the German occupiers, the Commercial Bank, survived under trusteeship management (treuhänderischer Verwaltung) throughout the war (Volkmann 2010, 285) and resumed its activities in the holding market after liberation. The new regulations imposed on notaries by the occupiers did not prevent them from continuing to offer domiciliation services. However, after the death of Paul Kuborn during the war, only the notary François Altwies continued to provide this service. While two major players - Kuborn and Levy Bank - disappeared, the market remained concentrated geographically around Boulevard Royal.

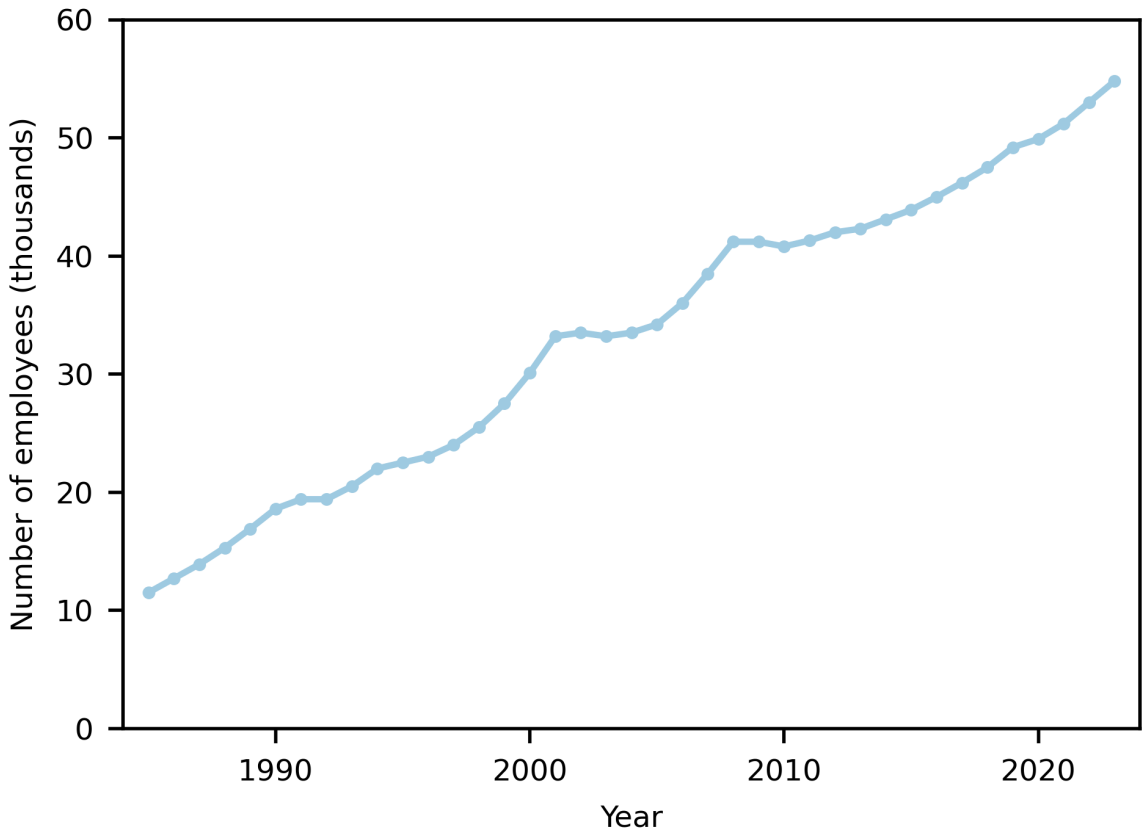
Specialisation and (partial) delocalisation

[Narrative layer]
From the 1960, we rely on a different ingestion of data: data has no longer been introduced manually but was processed digitally. Shell companies are no longer identified by their designation as "holding companies" but rather by the concentration of companies at specific addresses. The more companies registered at a single address, the higher the likelihood that they are shell companies. This criterion has been used by the EU Tax Observatory to map shell companies around the world (Aliprandi, Busschots, and Oliveira 2024). Until the 1980s, the historical center of the city of Luxembourg around the Boulevard Royal maintained its centrality: the geographical stability for something - shell companies - that did not need any physicality to exist is remarkable. The “Luxembourg’s Wall Street”, labeled like this by the Luxembourgish magazine Revue in 1965 (Duval, Gabellini, and Mouton 2023, 77) did only lose its centrality for the shell company market in the 1980s. Shell companies started to be registered outside of the traditional spatial boundaries. This expansion extends not only to the often-mentioned Kirchberg neighborhood, which became an important service area from the 1960s onward with the establishment of European institutions and financial service providers, and has already attracted some attention (Pesch 2014; Pauly 2018), but also to Limpertsberg.In the 1980s, the redevelopment of the Glacis plateau drew several banks and financial service providers to the area, at a time when many advocated for the development of Glacis, this "wasteland" separating the old town from Limpertsberg, as the site for the rapidly growing financial center (Miltgen and Hamilius Jr. 1987).

[Narrative layer]
A second shift became noticeable in the 2000s. The historic center has largely been abandoned, and new areas have been developed, such as Kirchberg, Belair-Limpertsberg, and Ban de Gasperich, a new district with a master plan in place since 2004 (Hesse 2013). In these new areas, the establishment of companies handling domiciliation often serves as a precursor and is considered an indicator of their viability. By the 2010s, this change was firmly established: of the 20 main addresses used by companies in Luxembourg, only three remain in the historic center of Luxembourg City, while the four most used addresses are now in Gasperich. This geographical shift reflects three key phenomena.

[Hermeneutics layer]
Firstly, there has been geographical expansion of the financial center, particularly towards Kirchberg and Ban de Gasperich. The growth of the people working there (Fig. 4) made it no longer possible to concentrate all the service providers inside the historical center confronted to a very high real estate demand.

In [8]:
```
from IPython.display import Image, display

display(Image("./media/employees-evolution.png"))
```



[Figure] **Fig. 4.**

[Hermeneutics layer]

Secondly, this expansion indicates a growing diversification in the financial service provider market. In 1999, a law regulated the activity of domiciliation agents for the first time, reserving it for professionals in the financial and insurance sectors, lawyers, auditors, and accountants (Ducat 2012). It introduced the obligation to know the real identity of the members of the company's governing bodies. This new regulation is leading to a reorganization of the domiciliation market, which is gradually being abandoned by banks, lawyers, and the Big Four, who leave this market to new players or outsource it. Thus, BIL, which has held a central position in domiciliation since the passing of the holding company law in 1929, decided in 2002 to create a new company, Experta, for managing domiciliation. The new subsidiary completely took over a department—Corporate Engineering—that previously existed within the bank ('Experta Luxembourg, la nouvelle filiale de Dexia BIL pour développer ses activités d'ingénierie financière' 2002). The large audit companies did the same, with PWC creating Alter Domus in 2004, and law firms like Arendt & Medernach creating Arendt Services in 2009. The new regulations, as well as the risks, notably reputational, associated with domiciliation, explain this outsourcing of an activity that since the 1930s had been an integral part of the activities of banks, notaries, and lawyers. (Thomas 2014). But there are also new players emerging who arrive in Luxembourg due to the market's specialization, a specialization that led to the creation of an association aimed at defending the sector's interests, the Luxembourg International Management Services Association, in 2004, which mainly gathers the major players. This process made the market less viable for smaller domiciliation companies. Indeed, while the top 20 most used addresses still included relatively small fiduciaries in the 1980s and 1990s, this is no longer the case from the 2000s onwards. However, unlike the banking world, lawyers, or even audit firms, which became objects of interest of social sciences in general and history in particular, both internationally and in Luxembourg, domiciliation remains largely an unknown area, even in the media.

[Hermeneutics layer]
Thirdly, the peripheralization of the domiciliation market is also linked to its being considered less prestigious despite its economic significance. In a 2014 study, economist Pierre Mallet estimated that the domiciliation sector employed around 2,600 people and generated 400 million euros in added value (0.9 percent of GDP). The sector is booming, thanks notably to investment funds. Between 2008 and 2012, the domiciliation business recorded annual growth of over eleven percent. Among the companies listed in 2016 as the main address providers, several rank among the top 50 employers in Luxembourg. However, unlike Chinese banks, which began investing in Luxembourg in the 2000s and primarily established themselves in the historic center — notably the Bank of China, which took over the historic BGL address and now enjoys the highest visibility on Boulevard Royal — this was not the case for domiciliation companies. The marginality of these companies is also reflected in the architecture of their buildings. In contrast to the prominence seen in the construction of banks in Luxembourg from the 1960s (Nottrot and Theis 2000; Duval, Gabellini, and Mouton 2023) and later lawyers and large law firms who enlisted renowned architects (Coubray Céline 2015b; 2015a), domiciliation companies opted for less notable

designs.

Global geographies

[Narrative layer]
Shell companies serve their purpose because they are embedded in larger global tax chains. Most Luxembourg holdings in the interwar period functioned adequately and did not require an additional layer of opacity: they did not yet operate like a matryoshka, the nested Russian dolls. In the 1930s, a Luxembourg shell company didn't not yet need shell companies from other countries to offer an effective product. However, several regions known as financial offshore centers were already mentioned in the Luxembourg company register. In general, the holdings market during the interwar period was oriented towards France and Belgium, which accounted for over 50% of the identified holdings. Following Luxembourg's exit from the German economic area after World War I (the end of the Zollverein, the withdrawal of German capital from the steel and financial sectors, etc.), the Grand Duchy became closely tied to the Franco-Belgian economy. (Leboutte, Puissant, and Scuto 1998): capital from these two countries took control of the Banque Internationale à Luxembourg. Belgium played a central role through the creation of the Banque Générale de Luxembourg, funded with Belgian money. In the early years of the holdings regime, Germany also held an important position—18% of the holdings identified in 1930 had ties to Germany—but this percentage dropped significantly after the National Socialists came to power in 1933. Between 1934 and 1939, the percentage of German-linked holdings fell to just 1%. While neighboring countries with strong economic relations with Luxembourg (France, Belgium, the Netherlands, Germany, and England) accounted for about 75% of the market, a country with limited economic ties to Luxembourg, namely Switzerland, began to play an increasingly significant role in this domiciliation landscape, rising from 5% in 1931 to 38% in 1937. By the end of World War I, Switzerland had become one of the most important players in the domiciliation market, offering a fiscally attractive holding regime as well as opportunities for concealment (Paquier 2001). This led to the emergence of more complex fiscal structures involving two offshore financial centers, such as Luxembourgish and Swiss shell companies intertwined to conceal French capital, providing even greater opacity. Other countries offering similar solutions, such as Liechtenstein, Panama, and Monaco, were (still) largely absent from the geography of the Luxembourg domiciliation market. After World War II, the two main domiciliation players, BGL and BIL, changed their registration strategies, no longer involving external individuals—a practice in the 1920s and 1930s that had allowed for giving a "nationality" to the holding, making it nearly impossible to identify the real beneficiaries as "national" entities.

[Narrative layer]
Using the second data series (1960-2016), the approach differs from that of the first series. To identify the geographies of global tax chains in which Luxembourg is involved, we based our analysis on locations defined as tax havens by James Hines. In 2010, he published a list

of 52 "treasure islands." (Hines Jr 2010). We cross-referenced this list /(adding Curaçao) with companies linked to these countries to determine Luxembourg's evolving position in the global geography of domiciliation.

Table 1 - Five most important represented countries in the “tax haven corpus”

1961–69	1970–79	1980–89	1990–99	2000–09	2010–16
Switzerland - 61.15%	Switzerland - 64.50%	Switzerland - 47.71%	Switzerland - 27.82%	Switzerland - 19.80%	Switzerland - 21.66%
Netherlands Antilles - 9.88%	Liechtenstein - 6.87%	Panama - 13.19%	Panama - 14.56%	British Virgin Islands - 11.49%	Jersey - 10.41%
Jersey - 5.86%	Jersey - 4.43%	Jersey - 5.37%	British Virgin Islands - 8.13%	Panama - 9.62%	Cayman Island - 9.76%
Bahamas - 6.45%	Netherlands - 3.63%	Hong-Kong - 4.81%	Ireland - 8.08%	Jersey - 8.69%	Ireland - 9.04%
Panama - 5.45%	Panama - 2.86%	Lebanon - 4.07%	Jersey - 5.72%	Ireland - 6.36%	British Virgin Islands - 5.64%

[Hermeneutics layer]
The data from Table 1 tells two stories. On one hand, it highlights the importance of certain continuities, emphasizing the significance of path dependencies established as early as the interwar period. Even though Switzerland's proportion fluctuates significantly over time, it remains the most important country throughout the 50 years covered by this corpus. Swiss banks began investing more heavily in Luxembourg starting in the 1970s, with the arrival of Union de Banques Suisses (UBS) in 1973 and Société de Banque Suisse (SBS) in 1974. Several Luxembourg banks opened branches in Switzerland: Compagnie Luxembourgeoise de Banque (a subsidiary of Dresdner Bank) since 1972, Kredietbank Luxembourg by acquiring Kredietbank Suisse, initially a subsidiary of KB Belgium established in 1970, since 1980 (with occasional branches in Basel and Lugano) (‘Expansion Im Auslandsgeschäft’ 1980), Banque Générale du Luxembourg operated in Zurich from 1982 to 2016, and BIL first in Lausanne (in 1985) and today in Geneva, Zurich, and Lugano. The Luxembourg market for domiciliation was closely monitored by the Swiss embassy in Luxembourg. This symbiosis between the two financial centers has existed since the interwar period. Given the wide variety of tax laws in Switzerland, it seemed interesting to present a more detailed view of the geographical links these companies have with different cantons.

Table 2 - Five most represented Swiss cantons among the Swiss corpus (1961-2016)

Geneva	Zurich	Zug	Bern	Ticino
34.81%	33.08%	12.49%	3.91%	2.15%

[Narrative layer]
The domiciliation sector in Luxembourg also followed international trends. Thus, the presence of the British Virgin Islands (BVI) in the Luxembourgish company register is linked to the rise of the Caribbean territory in this industry. The BVI were relatively late entrants to the shell company market but quickly assumed a pivotal role. In 1984, the “International Business Companies Ordinance” created an infrastructure that went beyond the low-level obligations that existed in other shell companies. It did so by eliminating the requirement for boards of directors to meet on tax-haven territory and by exempting investment earnings from income tax (Maurer 2000; Campbell 2021). The emergence of Ireland in the 1990s reflects the rise of a highly dynamic offshore financial center. With the creation of Dublin's International Financial Services Centre (IFSC) in 1987 and the adoption of a highly favorable regulatory framework (such as Section 110 of the Tax Consolidated Acts in 1997), Ireland quickly became a significant hub for tax planning for multinationals. (Murphy 1998; Allen 2016). While Ireland is often portrayed as a competitor to Luxembourg, the domiciliation market also highlights the interdependence and, therefore, some complementarity between the two European countries

[Narrative layer] Other offshore financial centers appear more sporadically. For example, Lebanon's strong presence is temporary and can be partly attributed to its development as a major financial center in the Middle East during the 1970s (Gregory 1976), which initially did not seem entirely disrupted by the outbreak of civil war from 1975 to 1990. Additionally, the oil shocks of 1973/74 and 1979 created a substantial petrodollar market, with Lebanon and Luxembourg emerging as key nodes: the former as a collection point, and the latter as a conduit for injecting these dollars into international markets. However, the prolonged war in Lebanon eventually led to the decline of Beirut as a financial hub.

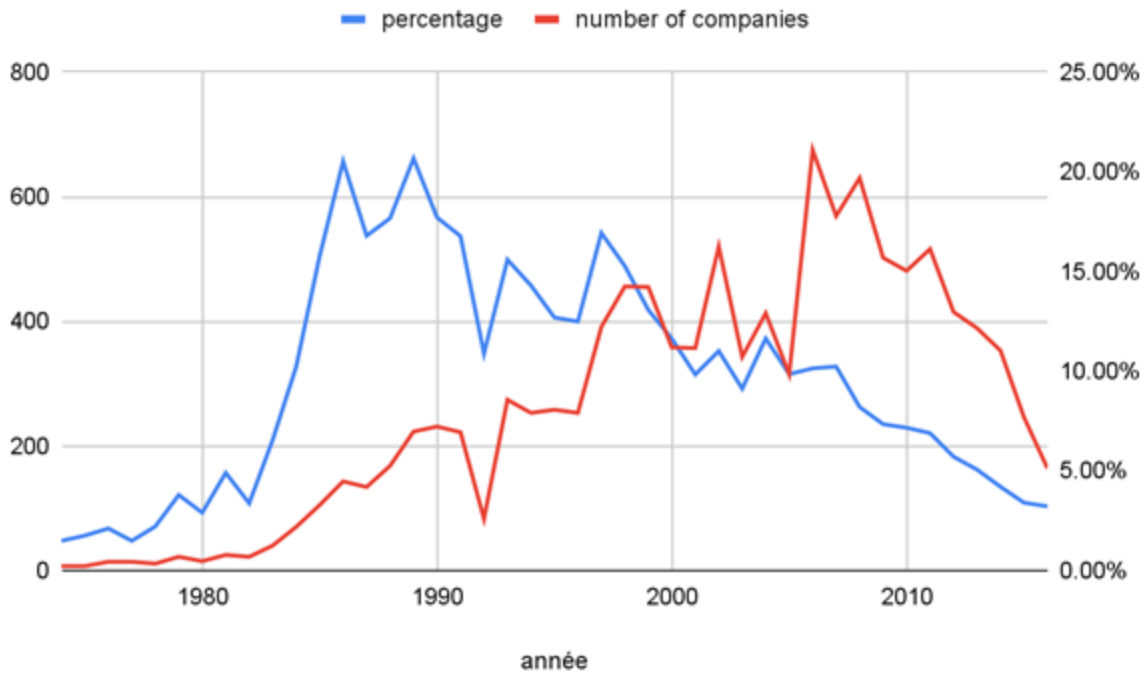
[Narrative layer]
But this overall global history of the financial center is very closely intertwined with local networks and professions, as the case studies of the relationship between Luxembourg and two offshore centers, Panama and Niue, show.

Panama, a long standing history

[Narrative layer]
In 1939, Panama appeared for the first time in the Luxembourg company register. The Luxembourg holding company Union Internationale de Placement, whose shareholders were French, was transferred to a Panamanian holding company. Since the construction of the

Panama Canal (1904-1914), the young republic had become a hub for financial innovation. In 1927, Panama introduced a new legal framework for companies, partly inspired by Delaware's legal regime. This new law explicitly provided a favorable environment for offshore companies (Barsallo Pérez 1994, 401). After World War II, although the number of Luxembourg holding companies with ties to Panama was not very high, the Central American country was regularly mentioned in the Luxembourg company register. The most well-known case was the use of Luxembourg by the IOS empire, a Panamanian company, to manage its investment funds in the 1960s, taking advantage of Luxembourg's light regulation and favorable tax regime (B. Majerus 2020; Calabrese 2023). Since the 1950s, Panama has emerged as a direct competitor to Luxembourg. During the budget debates in the Chamber of Deputies in 1955, socialist deputy Adrien van Kauvenbergh identified five countries that should be considered competitors in the field of "favored holding companies": Switzerland, Liechtenstein, Tangier in Morocco, Panama, and Curaçao. In the following years, this sentiment persisted. During the discussions initiated in the second half of the 1960s by Michel Debré, the French Minister of Economy and Finance, to harmonize taxation at the European level, a Luxembourg banker was quoted in La Vie Française, a French economic weekly, in February 1968 with the following remark when faced with the idea of reducing Luxembourg's advantages at the time: "Countries outside the Common Market will benefit — Switzerland, or even Bermuda or Panama. And even the United States, since Delaware is already competing with us.” Pierre Werner reiterated the same reasoning a year later in the Gazette de Lausanne in an article published in November 1969: "The European Community has no interest in destroying what has been built in Luxembourg for another reason as well: the operations that take place here would happen elsewhere anyway, for example in Panama or the Bahamas, but not within the Community, and would therefore escape its control. In December 1968, L’Echo de l’Industrie, a Belgian daily, published an opinion piece by Professor R. Vandeputte, in which he criticized Luxembourg’s position: "It is not healthy (...) that small states like the Grand Duchy or countries of insignificant size, such as Liechtenstein or Panama, owe part of their prosperity to policies that distort the normal application of tax regulations."

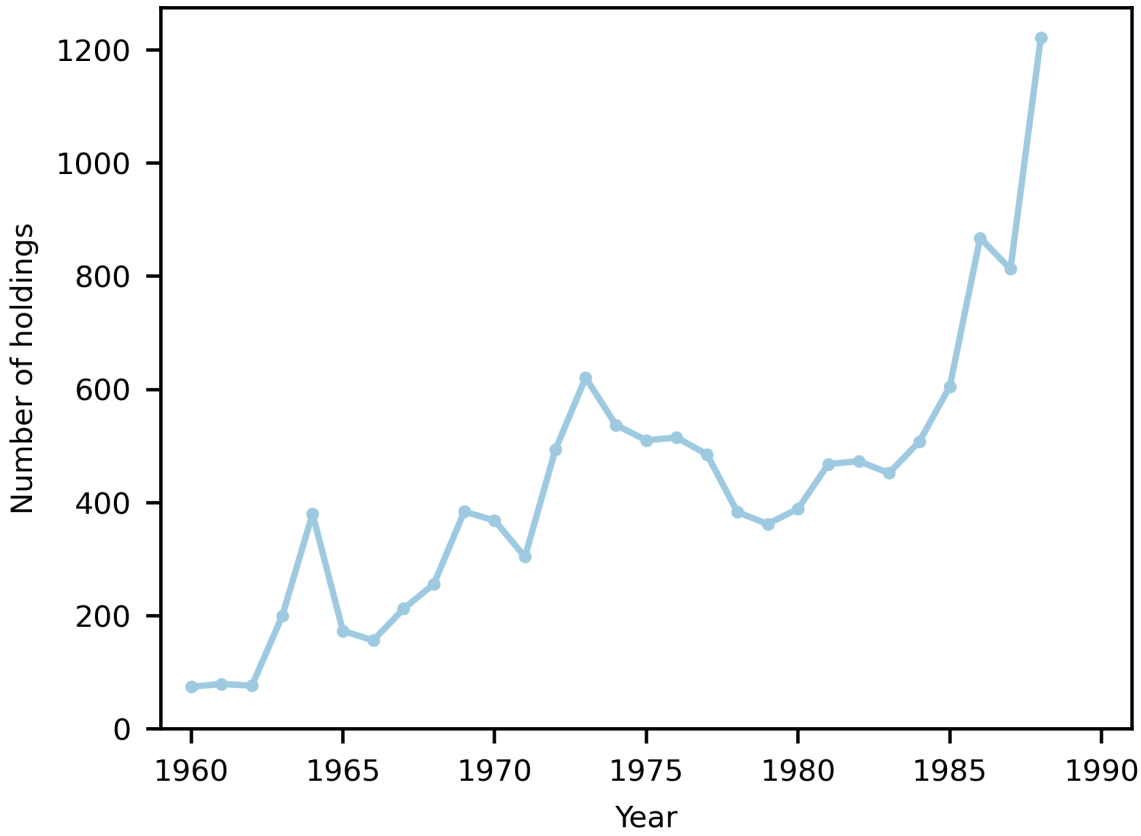
In [9]:
```
from IPython.display import Image, display

display(Image("./media/Fig H2.png"))
```



[Figure] Fig H2: Percentage inside the treasure island corps of companies with a reference to Panama

[Narrative layer]
It was from the 1980s onwards that the number and percentage of companies with ties to Panama increased significantly. The reasons for this take-off are numerous. While Panama was mainly presented as an offshore center focused on banking secrecy in the 1970s, ten years later, domiciliation became its main selling point. Starting from the second half of the 1970s, a more liberal political and economic regime was established. (Ardito-Barletta 1997; Conniff and Bigler 2019). In Luxembourg, domiciliation also emerged as an increasingly important niche starting from the second half of the 1960s, with significant acceleration in the late 1980s (Fig. 6).

In [9]:
```
from IPython.display import Image, display

display(Image("./media/holdings-evolution.png"))
```



[Figure] **Fig. 6.** Number of holdings created in Luxembourg between 1960 and 1988 (Statistiques historiques : 1839-1989 1990, 351)

[Hermeneutics layer]
This explains why Panamanian service providers opened branches in Luxembourg. The law firm Mossack Fonseca officially arrived in Luxembourg in 1988, eleven years after its founding in Panama (Poujol 2016). The notary, Marc Elter who managed Mossack Fonseca's registration, was one of the notaries specialized in Panama. His father, Prosper-Robert Elter, had already been one of the few notaries handling the domiciliation of companies in the late 1950s. The company "Mossack Fonseca & Co" aimed "to assist individuals and legal entities in the formation and domiciliation of companies": its office was located at 43 Boulevard Joseph II in an upscale neighborhood, just a few dozen meters away from, among others, the prestigious Luxembourgish law firm Elvinger Hoss Prussen. But by 1988, the Mossack Fonseca firm had already been active in the Luxembourg domiciliation market for seven years. In 1981, Jürgen Mossack, along with two other Panamanian businessmen, took over the administration of Intercommerce Holding, which was created in 1966 and had until then been managed by front men from KBL and belonged to a holding company created in 1954 and owned by Brussels-based companies. A year after Mossack Fonseca, Algocal opened an office in Luxembourg, on 13 boulevard Royal, with the help of the Luxembourgish lawyer company Bonn & Schmitt: Jaime Aleman and Alex Schmitt, the two main partners of the two law firms, were not only business partners but also maintained personal relationships: for instance, Alex Schmitt and his wife had attended the wedding of Jaime Aleman's daughter.

(Aleman, p. 643).

[Narrative layer]
Mossack Fonseca and Algocal were two relatively young law firms that are not part of the historical firms such as Arifa (est. 1914), IGRA (est. 1920), and Morgan & Morgan (est. 1923). Founded in 1977 for Mossack Fonseca and in 1985 for Algocal, they belong to a new generation of Panamanian lawyers. Among the historical firms, only Arifa opened a branch in 1993, with two Panamanian lawyers as directors and Evelyne Jastrow as their representative in Luxembourg.

[Narrative layer]
The actors who appeared in the Panama Papers (Raizer 2016) were highly biased because it was limited to the study of Mossack & Fonseca, which was indeed specialized in the domiciliation market but did not necessarily have a good reputation. The other major law firms were also present. Morgan & Morgan was represented by lawyer Patrick Houbert, Algocal by Alain Schmitt, but also by the fiduciary Lex Benoy.

[Narrative layer]
The longitudinal study that is made possible by the Luxembourgish company register sketch a more complex and complete image. Two groups of professions stand out, two groups that have so far scarcely appeared in the history of the Luxembourg financial center nor in the history of global tax chains: notaries and fiduciaries.

[Narrative layer]
In Luxembourg, the profession of notaries is highly regulated: their number is set by law. Often presenting themselves as mere authenticators of deeds, their role is much more central in the domiciliation industry. Through the case study of Panama, it becomes apparent that some notaries specialize in the domiciliation of companies. Regarding the Central American state, two notaries are implicated in the creation of almost a quarter of the companies with links to Panama: Joseph Elvinger and Jean Seckler. The former, close to the Luxembourg Liberal Party - in his youth, he was the Deputy Secretary General of the JDL ('Ils Dirigent La JDL' 1977) - has close links with the prestigious Luxembourgish law company Elvinger Hoss & Prussen: André Elvinger, one of the founder of EHP called him affectionately "our notary"(Elvinger, André 2017, 7). In 1998, Joseph Elvinger took over a notarial practice that had been in the domiciliation market since the interwar period. Tony Neuman was one of the four main notaries for the creation of holdings during the interwar period.(Calabrese and Majerus 2024). Neuman's successor, Charles Michels, was less active in this area, but his successor, Camille Hellinckx, dominated the market from the 1970s to the 1990s before Joseph Elvinger took over the practice from 1998 to 2014. Knowledge, networks and techniques are so transmitted from one generation to another. There is less known about the second important notary except that he was very well connected into financial networks. When a Luxembourg investment bank, Compagnie de Banque Privée Luxembourg, was established in 2006, Jean Seckler was among those involved, alongside Bob Bernard and

Norbert Becker, former managing partner of Arthur Andersen. (Poujol 2006).

[Narrative layer]
The second key players are the fiduciaries. Some larger Luxembourgish companies were integrated into larger international groups when what would become the “Big Four” established themselves in Luxembourg: Fiduciaire Générale du Luxembourg joined Touche, which later became Deloitte; Compagnie Fiduciaire Arthur Young; Interfiduciaire KMG; and PriceWater Fiduciaire Victor Steichen. However, other small fiduciaries continued to survive, particularly around the domiciliation market. Regarding Panama, Fiducenter stands out as an essential player. It was created in 1980 through the partnership of a Luxembourgish accountant, a Luxembourgish broker, and a Swiss fiduciary from the canton of Gsteig. Following the law on domiciliation in 1999, Fiducenter was one of the first companies to receive the now necessary accreditation to continue in this market (‘Dix Professionnels Agréés’ 2000). It opened an office in Cyprus in 2004, in Singapore in 2011, and in Malta. In 2013, it partnered with Oak, a service provider based in another country specializing in domiciliation, Guernsey. Fiducenter offered a nearly complete opacity shield: the companies it created were entirely staffed by people working at Fiducenter, thus providing no indication of the real beneficiaries of these offshore companies. Fiducenter closely worked with the two notaries most involved in the Panama market. The scandal surrounding the Panama Papers is not necessarily seen as a problem, as the scandal allows the market to be redirected, notably towards Cyprus, as explained by the head of the Cypriot office of Fiducenter in 2016.. Below Fiducenter, there were a small number of lesser-known fiduciaries that ensure this essential work of making company structures opaque through using Panamean companies among them Fidei Fiduciaire, founded in 1993 by the Belgian Bruno Beernaerts, and Fiduciaire Fernand Faber, established in 1952 and whose founder, Fernand Faber, had been president of the Order of Chartered Accountants of Luxembourg in the 1970s.

Niue, a short-living blip

[Narrative layer]
Niue is a small country in the South Pacific Ocean, covering 261.46 km^2 and home to around 1,600 inhabitants. It is closely associated with New Zealand, which handles several state functions on its behalf. Recently, Niue has made headlines as global warming directly threatens its existence, with rising sea levels jeopardizing the island's physical viability. The tourist office presents the island as “a place where it’s normal for complete strangers to wave at each other, all the time. It’s a place where nature hasn’t been broken… and things are 'the way they used to be'.”(‘The Official Website Of Niue Tourism’, n.d.) For a short time, this island appeared in the Luxembourgish company registers, being for two years the six most important treasure islands for Luxembourgish offshore companies (Table 3).

Table 3 - Percentage inside the treasure island corpus of companies with a reference to Niue

1995	1996	1997	1998	1999	2000	2001	2002	2003	2004	2005	2006
0%	0.59%	0.82%	3.38%	4.20%	4.78%	4.79%	2.35%	1.46%	1.49%	1.31%	0.68%

[Hermeneutics layer]
The reasons for this sudden surge are related to another offshore place where some Luxembourgish financial service providers have close relations with: Panama. It was in the early 1990s that the idea emerged within Panamanian business circles to use this island as a domiciliation space.The Niuean government was convinced to offer this new service. It was the Panamanian law firm Mossack Fonseca that drafted the legislative framework, inspired notably by the one used in the British Virgin Islands. Niue offered the advantage of avoiding the reputation issues that could arise from having a company domiciled in Panama. At the same time, having offices in a time zone suitable for the Asian market, particularly China and Russia, also proved to be advantageous. (Chambost 1999, 546–47; Fischer 2023, 197). Mossack Fonseca secured a 20-year monopoly for the domiciliation of companies, adopting a model of Foreign Professional Monopoly Agencies (FPMA), which had also been tried in other micro-states. (Van Fossen 2012). For a few years, this policy bore fruit: for three years, 10% of the Niuean government's revenue came from OFC fees. But quite quickly, the island of Niue appeared on tax haven lists. As early as 2001, among others, JP Morgan Chase Bank decided to stop using Niue. The pressure from international organizations was such that by 2002, the Niuean government committed to a policy aimed at limiting offshore legislation (Van Fossen 2012). Following this decision, Mossack Fonseca decided to shift its domiciliation market to the neighboring island of Samoa (Findley, Nielson, and Sharman 2014, 41). The events in Niue directly impacted the Luxembourg domiciliation market due to the Panamanian connection linking Niue and Luxembourg. As shown in the previous subchapter, several Panamanian law firms had established a presence in Luxembourg, including Mossack Fonseca, which had even opened an office in Luxembourg.

[Hermeneutics layer]
The relatively small number of companies linked to Niue and Niue's brief presence in the incorporation market offer the opportunity of a microhistory of domiciliation. Far from the large business law firms and international audit companies, we discover an almost invisible microcosm. Of course, there are notaries involved, with two particularly specialized in Niue: Norbert Muller (weighted degree of 1387, highest) and Gérard Lecuit (weighted degree of 503; fifth highest). Both appear regularly in the incorporation market without being among its most prominent figures. Around these two men are several women: Brigitte Siret (weighted degree of 487, sixth highest) and Sandra Vommaro with Norbert Muller, and Angela Palemburgi with Gérard Lecuit. It is not clear from the Mémorial C whether these women work directly for the two notaries, but they belong to a group of individuals, often women, who perform essential tasks of representation and concealment while occupying the lowest rung of the social ladder in the financial sector. They do not appear in specialized press like Paperjam or Agefi, and are rarely present on LinkedIn. Despite this invisibility, they play an essential role in the domiciliation market (see the portrait of Leticia Montoya, who

served as a sham director for at least 3,200 shell companies in the financial sector). Mossack&Fonseca: (Brinkmann, Obermaier, and Obermayer 2016)). A second group of people consists of men who seem to work more or less independently: François David (weighted degree of 503; fourth highest), listed as an "accountant" in the Mémorial C, but no other information has been found about him; Jérôme Guez, listed as a "financial director" in the Mémorial C, who states on X, "15 years as a fiduciary in Luxembourg"; and Jean-Marie Detourbet, listed as a "manager" in the Mémorial C. Of Belgian or French nationality, they were not deeply involved in the Luxembourg domiciliation market to leave more explicit traces there. Closely linked to these apporteurs d'affaires as they are labelled in French, men were fiduciaries and other financial service providers. For Niue, this were relatively small institutions such as LWM Corporate Services, linked to SEB Bank, a Swedish bank; Edmond Rothschild Bank; Luxfudicia, a fiduciary that employed five people in 2015; or Capitole Management Services, a domiciliation provider that went bankrupt in 2002. And finally the incorporation market also needs the intervention of lawyers. Three law firms are particularly represented: the Penning firm (Philippe (weighted degree of 121, 19th highest) and Jim), Chateaux Avocats (Jean-Pascal Cambier), and Lex Thielen (Lex Thielen, Philippe Stroesser). These are small firms, with the first not specializing in domiciliation, unlike the other two. While the names of Jürgen Mossack and Ramos Fonseca do not appear, but there is a multitude of sham directors linked to the network of these two lawyers, either in the British Virgin Islands, where they shifted part of their domiciliation market, or in Panama (Leticia Montoya (weighted degree of 1309; second highest) Juan Mashburn, Catalina Greenlaw, Darlene Bayne...).

[Hermeneutics layer]
In the mid-2000s, a trial in France revealed the profile of beneficiaries of these structures. In 2001, the company LX Partners was created before the notary Norbert Muller, owned by a company in Gibraltar and a company in Niue, represented by a man named Detourbet. In reality, it was a family of entrepreneurs from Metz trying to evade French VAT (Poujol 2014). Two rulings, one by the Court of Appeal on March 15, 2004, and another by the Administrative Tribunal on February 1, 2018, against the transmission of information to Belgian and French tax authorities, also indicate that in the 2000s, these arrangements involved people from neighboring countries of the Grand Duchy.

[Hermeneutics layer]
The Niuean case is revealing on two levels. On one hand, it shows the multiplicity of actors involved in making this system work between the Grand Duchy and a Pacific island, including notaries, sham persons in Luxembourg, the British Virgin Islands, and Panama, as well as banks, fiduciaries, and lawyers. On the other hand, these are actors who appear neither in the history of Luxembourg's financial center nor in the global histories of capitalism in the 20th century.

Conclusion

[Narrative layer]
Shell companies are a core function of Luxembourg's offshore financial sector. This activity had its roots in the 1930s. It remains important today, as evidenced by the reactions of Luxembourg's economic and political elites when the Unshell directive was announced (Klein 2022). This contribution attempts an initial mapping of this phenomenon by examining it at two different scales. Starting from the Luxembourgish addresses of shell companies, several phenomena emerge: the geographical dispersion of the financial hub, from the hypercenter around the boulevard in the 1930s to a multi-centered financial hub, though still concentrated in and around Luxembourg City in the 2010s. These addresses also reflect changes in the actors involved in domiciliation: initially a market dominated by banks, before transitioning to being managed by other service providers.

[Narrative layer]
By placing the Luxembourg domiciliation market within a global geography, certain geographical chronologies emerge: the central role of Switzerland throughout the 20th century for the Grand Duchy. The extreme responsiveness of the market to changes in capital coding is also evident: ten years after the "International Business Companies Ordinance," 12% of companies linked to tax havens are associated with the British Virgin Islands (BVI). At a time when communication had not yet been accelerated by the Internet, this implies a rapid spread of practices and the swift emergence of networks. These processes of transfers remain largely underexplored in this paper and will require further research, likely using additional sources.

[Narrative layer]
By intersecting these two scales, we can connect two mailboxes: one located at 16 Rue des Capucins, in the very center of Luxembourg City, the capital of a small state in the European Union, and P.O. Box 71 at 2 Centre Commercial Square in Alofi, the capital of a small island country in the South Pacific Ocean. Despite the one-sided perspective caused by the choice of sources—solely Luxembourgish—this focus highlights the importance of a localized infrastructure that has been largely overlooked by both Luxembourgish and international historiography: without these numerous "hidden helpers," there would be no offshore financial centers (Derix 2015; B. Majerus and Zenner 2020).

Bibliography

- Aliprandi, Giulia, Thijs Busschots, and Carlos Oliveira. 2024. *'Finding Shell Companies: Mapping the Global Geography of Shell Companies'*. Presented at the Global Tax Chains, Berlin.

- Allen, Kieran. 2016. *'Into the Limelight: Tax Haven Ireland'*. Irish Marxist Review 5 (16): 14–27.
- Anders, Jérôme. 1928. *Essai sur l'évolution bancaire dans le Grand-Duché de Luxembourg: étude historique et économique*. Luxembourg: Editions Luxembourgeoises.
- Ardito-Barletta, Nicolas. 1997. *'The Political and Economic Transition of Panama, 1978–1991'*. In *Democratic Transitions in Central America*, edited by Dominguez, Jorge and Marc Lindenberg, 32–66. Gainesville: University of Florida Press.
- Barsallo Pérez, Carlos Antonio. 1994. *'El secreto bancario en Panamá y España: un estudio comparativo'*. Universidad Complutense de Madrid. Facultad de Derecho. Departamento de Derecho Mercantil.
- Beckett, Paul R. 2023. *An Anatomy of Tax Havens: Europe, the Caribbean and the United States of America*. Berlin: De Gruyter. Link
- Bourbaki, Roger. 2016. *'End of Paradise ? Le Luxembourg et son secret bancaire dans les filets du multilatéralisme'*. Critique internationale 71 (2): 55–71. DOI
- Brinkmann, Bastian, Frederik Obermaier, and Bastian Obermayer. 2016. *'The World Of Sham Directors'*. Süddeutsche.De. 18 April 2016. Link
- Calabrese, Matteo. 2023. *'"The Grand Duchy's Fund Code": History of Investment Funds in Luxembourg from the Holding Act to the UCITS Normative (1929-1989)'*. PhD, University of Luxembourg.
- Calabrese, Matteo, and Benoît Majerus. 2024. *'Archaeology of a Treasure Island: Actors and Practices of Holding Companies in Luxembourg (1929–1940)'*. Contemporary European History 33:1398–1415. DOI
- Campbell, Aretha M. 2021. *Money Laundering, Terrorist Financing, and Tax Evasion*. London: Palgrave.
- Chambost, Édouard. 1977. *Guide des paradis fiscaux*. Collection Patrimoines. Paris: Tchou.
- ———. 1999. *Guide Chambost des paradis fiscaux*. 7th ed. Lausanne: Favre.
- Conniff, Michael L., and Gene E. Bigler. 2019. *Modern Panama: From Occupation to Crossroads of the Americas*. Cambridge: Cambridge University Press.
- Coubray Céline. 2015a. *'KPMG par Valentiny inauguré au Kirchberg'*. Paperjam. Link
- ———. 2015b. *'Tous sous le même toit'*. Paperjam, October. Link
- Davis, Nick. 2009. *'Tax Spotlight Worries Cayman Islands'*. BBC News, March. Link
- Derix, Simone. 2015. *'Hidden Helpers: Biographical Insights into Early and Mid-Twentieth Century Legal and Financial Advisors'*. European History Yearbook 16:47–62.
- *'Dix Professionnels Agréés'*. 2000. Agefi. Link
- Dörry, Sabine. 2016. *'The Role of Elites in the Co-Evolution of International Financial Markets and Financial Centres: The Case of Luxembourg'*. Competition & Change 20 (1): 21–36.
- Ducat, Alain. 2012. *'Avec Domicile Reconnu'*. Avec Domicile Reconnu, 98–102.
- Duval, Cécile, Marco Gabellini, and Victoria Mouton. 2023. *'La BGL, l'architecture d'un Siège Dans Une Ville En Mutation'*. Hémecht 75 (1): 49–79.
- Elvinger, André. 2017. *'Hommage à Notre Pionnier: Paul Elvinger'*.
- *'Expansion Im Auslandsgeschäft'*. 1980. Luxemburger Wort, August 29.

- *'Experta Luxembourg, la nouvelle filiale de Dexia BIL'*. 2002. Paperjam. Link
- Farquet, Christophe. 2010. *'Expertise et négociations fiscales à la Société des Nations (1923–1939)'*. Relations internationales, no. 142 (July): 5–21.
- Findley, Michael G., Daniel L. Nielson, and J. C. Sharman. 2014. *Global Shell Games*. Cambridge University Press.
- Fischer, Heinz-Dietrich. 2023. *100 Years of Pulitzer Prize Reporting on World Economy*. LIT Verlag Münster.
- Gregory, Pierre. 1976. *'La Place Financière De Beyrouth'*. Revue d'économie Politique 86 (2): 173–94.
- Guex, Sébastien. 2022. *'The Emergence of the Swiss Tax Haven, 1816–1914'*. Business History Review 96 (2): 353–72. DOI
- Hesse, Markus. 2013. *'Das «Kirchberg-Syndrom»: Grosse Projekte Im Kleinen Land'*. disP - The Planning Review 49 (1): 14–28. DOI
- Hesse, Markus, and Catherine Wong. 2019. *'Cities Seen through a Relational Lens'*. Geographische Zeitschrift, November. DOI
- Hines Jr, James R. 2010. *'Treasure Islands'*. Journal of Economic Perspectives 24 (4): 103–26.
- *'Ils Dirigent La JDL'*. 1977. Luxemburger Wort.
- *'In Search of Excellence'*. 2011. *Gold. The International Investment, Business & Finance Magazine of Cyprus*, no. 7 (October): 74–76.
- Klein, Thomas. 2022. *'Der Kampf gegen Briefkastenfirmen könnte Luxemburg hart treffen'*. Luxemburger Wort. Link
- *'Le groupe Maples se développe au Grand-Duché'*. 2018. Paperjam. Link
- Leboutte, René, Jean Puissant, and Denis Scuto. 1998. *Un Siècle d'histoire Industrielle, (1873–1973)*. Paris: SEDES.
- *'Loi Du 10 Août 1915 Concernant Les Sociétés Commerciales'*. 1915. *Mémorial Du Grand-Duché de Luxembourg*, Vol. 90. Link
- Majerus, Benoît. 2020. *'This Is Not a Scandal in Luxembourg'*. Entreprises et Histoire, no. 101: 75–87.
- Majerus, Benoît, and Benjamin Zenner. 2020. *'Too Small to Be of Interest, Too Large to Grasp?'*. European Review of History 27 (4): 548–62.
- Majerus, Nicolas. 1949. *Histoire Du Droit Dans Le Grand-Duché de Luxembourg*, Vol. 2. Luxembourg: Imprimerie Saint-Paul.
- Maurer, Bill. 2000. *Recharting the Caribbean*. Ann Arbor: University of Michigan Press.
- Miltgen, Daniel, and Jean Hamilius Jr. 1987. *'Auf Ewig Eine Öde'*. D'Lëtzebuerger Land, no. 37 (September): 13.
- Murphy, Laurence. 1998. *'Financial Engine or Glorified Back Office?'*. Area 30 (2): 157–65. DOI
- Nottrot, Iva, and Marc Theis. 2000. *Luxembourg: banks and architecture*. Luxembourg: G. Binsfeld.
- Paquier, Serge. 2001. *'Swiss Holding Companies from the Mid-Nineteenth Century to the Early 1930s'*. Financial History Review 8 (2): 163–82.

- Pauly, Michel. 2018. *'Luxembourg et Kirchberg: ville médiévale et capitale européenne'*. In *Urban Spaces and the complexity of Cities*, edited by Jean-Luc Fray. Wien: Böhlau. Link
- Pesch, Fernand. 2014. *Le Fonds d'urbanisation et d'aménagement du plateau de Kirchberg (FUAK)*. Esch-Sur-Alzette: Éditions le Phare.
- Philippart, Robert L. 2006a. *Luxembourg, de l'historicisme Au Modernisme*, Vol. 1. Steinsel: Editions Ilôts.
- ———. 2006b. *Luxembourg, de l'historicisme Au Modernisme*, Vol. 2. Steinsel: Editions Ilôts.
- Poujol, Véronique. 2006. *'Double Lion Rouge'*. D'Lëtzebuerger Land, 9.
- ———. 2007. *'Apporteurs d'affaires'*. D'Lëtzebuerger Land, November: 11.
- ———. 2014. *'La Chasse Aux Fraudeurs'*. Paperjam, March.
- ———. 2016. *'Quand Mossack Fonseca dénonçait la fraude fiscale au Luxembourg'*. Paperjam. Link
- Raizer, Thierry. 2016. *'Panama Papers: à l'ombre de l'offshore mondiale'*. Paperjam. Link
- *'Règlement Grand-Ducal Du 9 Janvier 1961'*. n.d. Accessed 23 September 2019. Link
- Sinnig, Julia, and Dirk A. Zetzsche. 2023. *'The Impact of the Draft "Unshell Directive"'*. SSRN. DOI
- *Statistiques historiques: 1839–1989*. 1990. Luxembourg: STATEC.
- *'The Official Website Of Niue Tourism'*. n.d. Accessed 10 June 2024. Link
- Thomas, Bernard. 2014. *'How the West Was Won'*. Lëtzebuerger Land, June. Link
- Van Fossen, Anthony B. 2012. *Tax Havens and Sovereignty in the Pacific Islands*. St Lucia: Univ. of Queensland Press.
- Volkmann, Hans-Erich. 2010. *Luxemburg Im Zeichen Des Hakenkreuzes*. Paderborn: Schöningh.
- Walther, Olivier, Christian Schulz, and Sabine Dörry. 2011. *'Specialised International Financial Centres and Their Crisis Resilience'*. Geographische Zeitschrift 99 (2/3): 123–42.
- Watteyne, Simon. 2023. *Lever l'impôt En Belgique*. Bruxelles: CRISP.
- Weber, Josiane. 2013. *Familien der Oberschicht in Luxemburg*. Elitenbildung & Lebenswelten 1850–1900.
- Weitzman, Hal. 2022. *What's the Matter with Delaware?*. Princeton: Princeton University Press. DOI
- Singer-Vine, Jeremy, and Samkit Jain. 2025. "pdfplumber." GitHub repository. https://github.com/jsvine/pdfplumber
- Hoffstaetter, Samuel. 2025. "pytesseract." GitHub repository. https://github.com/madmaze/pytesseract

In [ ]: